CareerForge

AI-Powered Career & Academic Intelligence Platform

Software Engineering Report — System Requirements, Architecture,
Data Model, Use Cases & Sequence Design

Next.js **TypeScript** **React Native** **Supabase** **Prisma** **LLM API**

Capstone / Final Year Software Engineering Project

Document Type: System Design & Requirements Report

Version 1.0 — August 2026

Table of Contents

1. Executive Summary 3
2. Problem Statement & Motivation 4
3. Project Objectives & Scope 5
4. Core Modules Overview 6
5. Functional Requirements 7
6. Non-Functional Requirements 11
7. System Architecture 12
8. Technology Stack 14
9. Database Design (ERD) 15
10. Use Case Model 18
11. Sequence Diagrams 20
12. System Data Flow 23
13. API Design 26
14. Security, Session & Secrets Management 28
15. AI Integration Reliability 29
16. Job-Market Data & Application Channels 31
17. Mobile App (React Native / Expo) Notes 32
18. Testing, CI/CD & Quality Assurance 33
19. Development Roadmap 35
20. Getting Started — How to Build/Run 36
21. Future Enhancements 37
22. Risks & Mitigation 38
23. Conclusion 39

1. Executive Summary

CareerForge is an AI-powered career and academic intelligence platform aimed at university students and recent graduates who lack a clear, data-driven path from where they are academically to the career they want. Rather than being a single-purpose CV builder, CareerForge unifies career discovery, skill-gap analysis, academic planning, job-market intelligence, personalized learning roadmaps, CV analysis, and automated portfolio generation into one closed-loop system — and, at its core, an **AI Application Agent** that discovers suitable jobs, tailors a CV and cover letter for each one from the student's *verified* profile data only, fills in application forms, and manages submission and outcome tracking through to interview or offer.

The platform is built on a unified **Next.js + TypeScript** backend/frontend (web), a **React Native (Expo)** mobile client communicating over a REST/JSON API, a **PostgreSQL** database accessed through **Prisma ORM**, and an **LLM API** integration layer for reasoning tasks such as generating plain-language explanations of deterministically-computed scores, CV/cover-letter tailoring, and (as a stretch goal) an in-app AI assistant. This report documents the functional and non-functional requirements, system architecture, entity-relationship model, use cases, sequence flows, and a phased delivery plan suitable for a capstone-scale software engineering project.

> **What "automatic application" means:** CareerForge is designed to automate the job-application process end-to-end where the target job platform provides a permitted and supported application API/integration (Tier 1, Section 16.2). On platforms without a supported submission API, CareerForge automatically prepares the tailored CV, cover letter, and application fields but requires one explicit user confirmation before submission. The system never bypasses CAPTCHA, bot detection, authentication controls, or platform restrictions to force automation where it isn't supported.

> **The autonomous component:** the AI Application Agent (M7) is the part of CareerForge that can act without a per-application prompt. When the user enables auto-apply and a job satisfies the user's predefined eligibility, exclusion, and match-score criteria (Section 4, `ApplicationAgentSettings`, Section 9.3), the Agent can independently prepare and submit the application through a supported Tier 1 integration. For unsupported platforms, the Agent stops before submission and requests explicit user confirmation (Tier 2) or hands over a ready-to-send bundle (Tier 3). So: yes, the AI can apply on its own — but only where doing so is technically supported and the user has explicitly authorized it.

> **Key differentiator:** CareerForge does not give generic advice ("learn Docker") and it does not stop at recommending jobs — it can act on them, within limits the user controls. Every recommendation and every generated application artifact is derived strictly from the student's verified profile and the job posting itself, using deterministic scoring for anything measurable (Section 15.3) and the LLM only to explain results and generate language — for example, the system may report "Docker appears in 63% of analyzed job postings matching your selected career and filters" (Section 16.1), not as a universal labor-market statistic. The AI Application Agent is contractually forbidden from inventing a skill, role, or qualification the student never entered (Section 15.5), and the system closes the loop by tracking real outcomes (applications, interviews, offers, rejections) and adapting future matching, CV tailoring, and career recommendations accordingly (Section 12).

2. Problem Statement & Motivation

University students and fresh graduates commonly face a fragmented and confusing transition from academic study to employment. Typical unanswered questions include:

- What career should I pursue given my background?
- Am I actually qualified for a specific job?
- What skills am I missing, and which of them matter most?
- Which courses should I prioritize before graduation?
- What projects would meaningfully strengthen my profile?
- Is my CV good enough — and good enough for *which* job?
- Why do I keep getting rejected after applying?
- How do I build a portfolio when I don't know what belongs in it?

Today, answering these questions requires students to piece together information from many disconnected platforms (LinkedIn, job boards, course catalogues, generic CV checkers, career-advice articles). None of these tools share context about the student, so none of the advice is truly personalized, and there is no feedback loop connecting "what I did" to "what I should do next".

2.1 Proposed Solution

CareerForge consolidates profile data (education, skills, experience, projects, certifications, career goals) into a single model, then layers career analysis, academic planning, job-market intelligence, and CV/portfolio tooling on top of it — so every recommendation is explainable and grounded in the student's actual data and real market demand.

3. Project Objectives & Scope

3.1 Objectives

- Build a single profile model that captures education, skills, experience, projects, certifications, and job-application preferences as the one **verified source of truth** for everything the AI generates.
- Compute career-match and job-match scores deterministically from verified profile data, and use an LLM only to explain those scores and generate language (CV/cover-letter text, feedback) — never to invent the score itself.
- Quantify skill gaps against both a target career and an aggregated dataset of job postings representing real-world job requirements (Section 16.1).
- Turn skill gaps into a concrete, time-boxed learning roadmap and project recommendations.
- **Develop an AI Application Agent capable of autonomously identifying suitable jobs, preparing job-specific application materials, completing supported application forms, submitting applications through permitted integrations when the user has enabled automatic submission, and continuously tracking application outcomes** — without ever fabricating a qualification the student didn't provide, and without exceeding the limits, exclusions, and consent the user has configured (Section 4, M7).
- Track real application outcomes and feed them back so future matching, CV tailoring, and career recommendations improve.
- Automatically generate a shareable portfolio from the same profile data — no duplicate data entry.
- Deliver the system across both web (Next.js) and mobile (React Native / Expo) using one shared, versioned backend/API.

3.2 In Scope (Capstone Delivery)

The seven **core modules** below are treated as required for the capstone deliverable (numbered M1–M7 in Section 4):

1. Profile & Onboarding
2. Career & Skill Analysis
3. Academic / Course Planner
4. Job Discovery & Matching
5. Personalized Learning + Project Roadmap
6. CV Analyzer + Portfolio Builder
7. **AI Application Agent** — CV adaptation, cover-letter generation, application-form completion, tiered submission, and outcome tracking (Section 4, 12, 15.5)

3.3 Out of Scope for v1 (Stretch Goals)

- Conversational AI Career Assistant (free-form Q&A over the user's profile) — **M9**
- Graduation / "what-if I fail this course" simulation engine — **M8**
- Job-market analytics dashboards (aggregate demand charts across postings)
- Dedicated Python/FastAPI ML microservice for custom NLP models
- Additional Tier 1 ATS integrations beyond the one built for v1 (Section 16.2)
- Public job-board API ingestion (Section 16.1)

The Application Tracker, previously a stretch goal, is now part of the core AI Application Agent (M7) — see Section 4.

Rationale: this keeps the first release realistically finishable while leaving clear, already-modeled extension points (see Section 19, Development Roadmap) for every stretch feature.

3.4 MVP Scope Boundary

> **Job data source (v1):** for the capstone MVP, job postings are provided through an admin-managed dataset and a controlled **Mock ATS** environment (Section 3.5) — not by scraping or depending on a live third-party job board. External job-board API integrations are a future enhancement (Section 16.1, Section 21), not a v1 dependency. This is a deliberate choice, not a gap: it keeps job data legally clean and every capstone demonstration reproducible on demand.

> **Overall MVP boundary:** the MVP prioritizes the complete student journey — profile through career analysis, skill gap, academic planning, job discovery/matching, learning roadmap, CV/portfolio, and the full AI Application Agent pipeline — plus **one controlled Tier 1 application integration** (the Mock ATS, and optionally one real sandbox ATS if time allows). Additional real-world ATS integrations, advanced analytics dashboards, and the M8/M9 stretch modules are explicitly deferred (Section 21). This boundary exists so the project can be implemented at production quality within a capstone timeline, rather than at prototype quality across a much larger surface. The engineering risks this scope decision and the autonomous Agent introduce — and how each is mitigated — are collected in one place in Section 22, Risks & Mitigation.

3.5 Mock ATS / Controlled Application Environment

The Mock ATS is a **core demonstration component of the v1 MVP**, not an afterthought — it is what makes the AI Application Agent's autonomous-submission claim actually demonstrable without depending on a production job platform's cooperation during a capstone defense. It reproduces an application form and a submission API inside CareerForge's own infrastructure, and is treated as a genuine Tier 1 integration by the Submission Router (Section 7.2, Section 16.2) — same code path, same idempotency and failure-handling behavior as a real ATS. It is what lets the full sequence run live: *AI finds job → evaluates job → tailors CV → generates cover letter → fills form → submits → tracks application*.

4. Core Modules Overview

The system is organized into nine functional modules. The first seven are the required capstone core; the remaining two are stretch goals built on the same data model.

M1 — Profile & Onboarding CORE

Captures education, skills, experience, projects, certifications, interests and career goals, plus the user's **job-application preferences** — preferred locations, remote/hybrid/on-site mode, employment type, minimum salary, experience level, work-authorization/sponsorship preference, preferred industries, and daily application limits. This is the single, **verified** source of truth every other module — including the AI Application Agent — reads from and is never allowed to add to on its own.

M2 — Career & Skill Analysis CORE

Computes ranked career matches using a **deterministic scoring engine** based on verified profile data, then uses the LLM to explain the results in plain language. It also identifies and prioritizes skill gaps as Critical, Important, or Optional.

M3 — Academic / Course Planner CORE

Tracks degree progress (credits completed vs. required) and recommends which remaining courses to prioritize based on the student's chosen career target.

M4 — Job Discovery & Matching CORE

Aggregates active, non-expired job postings, extracts required skills, and computes a deterministic compatibility score per job (*Apply* / *Not yet*) as well as a "% of analyzed postings requiring skill X" market-demand view (Section 16.1). Every job above the student's match threshold is handed to M7's eligibility filter as a candidate to act on.

M5 — Learning Roadmap & Project Recommendations CORE

Converts the skill gap into a month-by-month roadmap and recommends specific projects designed to demonstrate multiple missing skills at once.

M6 — CV Analyzer + Portfolio Builder CORE

Computes a deterministic CV-to-job compatibility score (required skills / experience / projects / keywords / education) against a target job, while the LLM provides detailed explanations and actionable improvement suggestions, and auto-generates a public portfolio page from the same profile data. Versioned baseline CVs are the starting point M7 tailors per job (Section 5.6, FR-6.6).

M7 — AI Application Agent CORE

The end-to-end pipeline in Section 12, built from nine internal components (Section 7.2): a **Job Eligibility Filter** and **Application Policy Engine** that check the job against the user's `ApplicationAgentSettings` (threshold, daily limit, exclusions, duplicate check) before anything is generated; a **CV Tailoring Service** and **Grounding Validation Service** that adapt the CV and cover letter from verified data only and reject anything that fails the anti-fabrication check (Section 15.5); an **Application Form Mapper** that auto-fills what it safely can; a **Submission Router** that sends the job through Tier 1 auto-submit, Tier 2 assisted, or Tier 3 manual (Section 16.2); and an **Application Tracker** plus **Outcome Analysis Service** that record status and feed results back into M2, M4, and M5 (FR-7.8). Every step is grounded, policy-checked, and logged — it never invents a qualification the student didn't provide, and never acts outside the limits the user configured. Acceptance criteria: Section 5.11. Key risks and their mitigations: Section 22.

M8 — Graduation / Course-Failure Simulator STRETCH

Models course prerequisite chains as a graph and simulates the downstream effect of failing or delaying a course on graduation timeline.

M9 — AI Career Assistant STRETCH

A conversational interface over the student's own profile/data ("Should I learn Docker or Kubernetes first?"), answered with context rather than generic advice.

Admin Dashboard CORE (Admin-facing)

Not a student-facing module but a required interface for the Admin actor (Section 10): manage job postings, maintain the skill/course taxonomy, configure application-source submission tiers, and view platform analytics — the administrative use cases UC-10 to UC-12 (Section 10.1), numbered separately from the student use cases UC-1 to UC-9. Needed from v1 since M4 (Job Discovery & Matching) has nothing to match against without it.

5. Functional Requirements

Requirements are grouped by module and tagged **FR-x.x**. Priority follows MoSCoW: **M**ust, **S**hould, **C**ould.

5.1 M1 — Profile & Onboarding

ID	Requirement	Priority
FR-1.1	User can register/log in via email+password or OAuth, backed by Supabase Auth.	M
FR-1.2	User can create/edit a profile: bio, career goal, location, interests.	M
FR-1.3	User can add/edit/delete education history (institution, degree, dates, GPA).	M
FR-1.4	User can add/edit/delete skills with a self-rated proficiency level.	M
FR-1.5	User can add/edit/delete work experience and personal/academic projects.	M
FR-1.6	User can add certifications with issuer and date earned.	S
FR-1.7	User can define job-application preferences and constraints, including location, work mode (remote/hybrid/on-site), employment type, experience level, salary preference, and other relevant filters.	M
FR-1.8	User can define an automatic-application match-score threshold and a maximum number of applications the Agent may submit within a specified period.	M
FR-1.9	User can exclude specific companies, job types, locations, or other job categories from automatic application.	S

5.2 M2 — Career & Skill Analysis

ID	Requirement	Priority
FR-2.1	The system computes a deterministic career-match score using predefined weighted criteria based on the user's verified profile and career requirements. The LLM generates a plain-language explanation of the computed score and the main factors contributing to the match.	M
FR-2.2	User can select a target career and view a skill-gap table (have / missing / partial) against that career's typical requirements.	M
FR-2.3	System classifies gaps as Critical / Important / Optional.	M
FR-2.4	Career match results and gap analyses are persisted and re-computed on profile change.	S

5.3 M3 — Academic / Course Planner

ID	Requirement	Priority
FR-3.1	User can enter enrolled/completed courses with grade and semester.	M
FR-3.2	System computes degree progress (% and credits completed vs. required).	M
FR-3.3	System recommends which remaining/elective courses to prioritize based on the user's target career.	S

Course-failure simulation is not an M3 requirement — it is specified separately as M8 (Section 5.8) to keep it clearly scoped as a stretch module rather than an extension of the core planner.

5.4 M4 — Job Discovery & Matching

ID	Requirement	Priority
FR-4.1	System ingests job postings (Section 16.1) with parsed required-skill lists, source, external job ID, retrieval timestamp, and submission tier.	M
FR-4.2	System computes a deterministic per-job compatibility score against the user's skill profile and labels it Apply / Not yet.	M
FR-4.3	System aggregates skill frequency across the currently available, normalized job-posting dataset for a target career and filters (e.g. "Docker appears in 63% of analyzed job postings matching your selected career and filters") — presented as a dataset statistic, not a universal labor-market figure.	S
FR-4.4	User can filter/browse jobs and see missing-skill and missing-experience explanations per posting.	M
FR-4.5	System excludes or flags stale/expired postings (past their retrieval freshness window or application deadline) from active matching and from the AI Application Agent's eligible-job set.	M

5.5 M5 — Learning Roadmap & Project Recommendations

ID	Requirement	Priority
FR-5.1	System converts a skill gap into a month-by-month learning roadmap (topics/skills per month).	M
FR-5.2	User can mark roadmap items complete; the roadmap re-orders/adapts on progress.	S
FR-5.3	System recommends specific projects designed to close multiple skill gaps at once, with a rationale ("demonstrates Java,	M

	Docker, AWS").	

5.6 M6 — CV Analyzer + Portfolio Builder

ID	Requirement	Priority
FR-6.1	User can upload a CV (PDF/DOCX); system parses it into structured sections.	M
FR-6.2	The system computes a CV-to-job compatibility score using deterministic criteria based on required skills, experience, projects, education, and keywords, while the LLM provides detailed explanations and actionable improvement suggestions.	M
FR-6.3	System lists missing keywords/skills and gives section-level, actionable rewrite suggestions (not generic tips).	M
FR-6.4	System auto-generates a public portfolio page (About, Projects, Skills, Experience, Education, Certifications, Contact) from existing profile data.	M
FR-6.5	User can choose between portfolio templates and toggle public/private visibility.	S
FR-6.6	The system maintains versioned copies of the user's baseline CV and every job-specific tailored CV, allowing the exact document used for each application to be retrieved later.	M

5.7 M7 — AI Application Agent CORE

ID	Requirement	Priority
FR-7.1	For every job M4 marks at or above the user's match threshold, the system (re-)confirms compatibility using the deterministic scorer (Section 15.3) before acting on it.	M
FR-7.2	System generates a CV tailored to that specific job — reordering, re-weighting and rewording content — using **only** facts already present in the user's verified profile (Section 5.1). It must never add a skill, employer, title, project, certification, or metric the user did not enter.	M
FR-7.3	System generates a job-specific cover letter when the posting requires one, grounded in the same verified-data constraint as FR-7.2.	M
FR-7.4	The system automatically fills application fields when the required information is explicitly available in the verified user profile. If a question requires information that is unavailable, ambiguous, subjective, or cannot be safely derived from the profile, the system must leave the field unanswered and request user input rather than guessing.	M
FR-7.5	System submits the application automatically only on platforms with a supported API/integration (Tier 1, Section 16.2) and only when the user has enabled auto-submit — globally or per job — above their chosen match threshold; on platforms without a submission API (Tier 2/3) the system prepares the ready-to-send application and requires one explicit user action to send it.	M
FR-7.6	Every application record stores: the job posting, the exact submitted CV version, the cover letter (if any), submission date, submission channel (auto/assisted/manual), and current status.	M
FR-7.7	System tracks status across the application lifecycle (draft → submitted → viewed → interview → offer / rejected → withdrawn) via user updates and, where a platform exposes one, status-polling or webhook integration.	M
FR-7.8	Application outcomes are stored and analyzed to identify patterns in the user's application performance. The system may use these outcomes to adjust future recommendations and prioritization (e.g. de-prioritizing a career path with a persistently low response rate), while changes to the deterministic scoring algorithm itself remain controlled by system configuration and are not automatically modified by the LLM.	S
FR-7.9	**The system must never fabricate qualifications, experience, education, projects, certifications, skills, employment history, or measurable achievements.** Every factual claim about the applicant in a generated CV, cover letter, or application answer must be traceable to verified profile or CV data. Job-specific facts such as the company name, job title, and job requirements may be sourced from the verified job posting (Section 15.5).	M
FR-7.10	User can review and edit any generated CV/cover letter/answer before it is used, and can enable, restrict, or revoke auto-submission consent at any time; auto-submit is opt-in, never a silent default.	M
FR-7.11	User can view response/interview rate and system-surfaced likely reasons for a low rate (CV match too low, missing skills, applying above level, weak portfolio); the system tracks application-to-response rate, response-to-interview rate, interview-to-offer rate, rejection rate, and average time to response.	S
FR-7.12	The AI Application Agent must check whether the user has already applied to the same job or an equivalent job posting before creating or submitting an application. Duplicate applications must be prevented or explicitly flagged for user review.	M
FR-7.13	The AI Application Agent must enforce user-configurable application limits, including a maximum number of automatic submissions per day and a minimum match score required for automatic submission (`ApplicationAgentSettings` , Section 9.3).	M
FR-7.14	The user can define jobs or employers that the Agent must never apply to, and these exclusions must be checked before any application is submitted.	M

5.8 M8 — Graduation / Course-Failure Simulator STRETCH

ID	Requirement	Priority
FR-8.1	The system can simulate the effect of failing or delaying a prerequisite course and display the resulting impact on the	C

	student's expected graduation timeline.	

5.9 M9 — AI Career Assistant STRETCH

ID	Requirement	Priority
FR-9.1	User can ask free-form questions; the assistant answers using the user's own profile/roadmap/job data as context (RAG-style prompt assembly), not generic advice.	C

5.10 Admin Dashboard CORE (Admin-facing)

ID	Requirement	Priority
FR-A.1	Admin can create, edit, deactivate, and delete job postings.	M
FR-A.2	Admin can define and update the required skills, career categories, course taxonomy, and experience levels used by the matching engine.	M
FR-A.3	Admin can configure the application submission tier associated with each supported job source or ATS.	M
FR-A.4	Admin can view basic platform statistics including number of registered users, jobs, applications, interviews, offers, and rejections.	S
FR-A.5	Admin actions are authenticated, role-restricted, and recorded in an audit log (Section 9.3, `AuditLog`).	M

5.11 Acceptance Criteria — Core Modules

Each core module's functional requirements (5.1–5.10) are testable individually, but an evaluator (or the team, during a demo) needs a single pass/fail checklist per module. These are the acceptance criteria Section 18's test suite is written against.

M1 — Profile & Onboarding is successful if:

- A user can complete registration, build a profile, and set application preferences (FR-1.7–1.9) without needing any other module.
- RLS prevents one authenticated user from reading or writing another user's profile data (Section 9.4).

M2 — Career & Skill Analysis is successful if:

- The deterministic scorer produces the same career-match score for the same profile input (no LLM variance in the number itself).
- The LLM explanation references the actual factors the scorer used, not generic text.
- Skill gaps are classified into Critical/Important/Optional and persisted with the match.

M3 — Academic / Course Planner is successful if:

- Degree progress (% and credits) is calculated correctly from entered course records (FR-3.2).
- Recommended courses are actually drawn from the gap identified by M2 for the user's target career.

M4 — Job Discovery & Matching is successful if:

- Only active, non-expired postings (FR-4.5) are returned as matches.
- The compatibility score is reproducible for the same user/job pair.
- Postings below the match threshold are labeled "Not yet," not hidden without explanation.

M5 — Learning Roadmap & Project Recommendations is successful if:

- Every roadmap item traces back to a specific skill gap identified by M2.
- Marking an item complete updates the roadmap without requiring full regeneration.

M6 — CV Analyzer + Portfolio Builder is successful if:

- The CV-to-job score is computed deterministically from parsed CV content, not the LLM.
- Every baseline CV version is retrievable later (FR-6.6).
- The generated portfolio page reflects only data already in the profile — no manual re-entry.

M7 — AI Application Agent is successful if:

- The Agent finds an eligible job and the match score is at or above the user's configured threshold.
- The duplicate-application check passes before any generation begins (FR-7.12).
- User-defined exclusions (companies, job types, locations) are respected (FR-7.14).
- The daily application limit is not exceeded (FR-7.13).
- The tailored CV and cover letter are generated only from verified profile information.
- Grounding validation passes before any document is attached to an application (Section 15.5).
- The application form is completed, with unresolvable fields left for the user rather than guessed (FR-7.4).
- For a Tier 1 job, the Mock ATS (or sandbox ATS) receives the application and returns an application ID, which is stored on the `Application` record.
- The application appears in the tracker (UC-9) with the correct status and submission channel.

> **In one sentence:** M7 is successful if the Agent identifies an eligible job, verifies the user's constraints, prevents duplicates, generates grounded application materials, completes the Mock ATS form, submits successfully, and records the resulting application status.

Admin Dashboard is successful if:

- An admin can create a job posting with a submission tier (FR-A.3) and it is immediately eligible for M4 matching.
- A non-admin account is rejected by both the API role check and RLS when attempting any `/api/v1/admin/*` action (Section 14.3).

6. Non-Functional Requirements

Category	Requirement
Performance	Standard CRUD API responses < 300ms (p95). LLM-backed endpoints (career match, CV analysis) should stream or return within 8s, with a loading state on both web and mobile clients.
Scalability	Stateless Next.js API routes so the app can scale horizontally on Vercel; the underlying Supabase Postgres instance is sized/indexed to handle a growing job-postings table (indexed on skills/career fields).
Security	Auth delegated to Supabase Auth (email+password and OAuth providers, JWT-based sessions); Postgres **Row Level Security (RLS)** policies enforce that a user can only read/write their own rows (profile, skills, applications, roadmap) directly at the database layer, in addition to API-level checks; uploaded CV files stored in private Supabase Storage buckets with signed URLs.
Privacy	CVs and profile data are personal data — never exposed publicly except the fields the user explicitly marks visible on their portfolio.
Usability	Career-match and skill-gap results must always include a plain-language "why", not just a score — this is a core UX requirement, not cosmetic.
Reliability	If the LLM API call fails or times out, the system degrades gracefully (cached last result, or a clear retry state) rather than blocking the whole page.
Maintainability	Single Prisma schema as the source of truth for the data model; shared TypeScript types between API routes and web client.
Portability	Mobile (React Native / Expo) and web (Next.js) both consume the same versioned REST/JSON API — no client-specific backend logic.
Auditability	Every AI-generated recommendation (career match, CV feedback, roadmap) is stored with a timestamp and the input snapshot it was generated from, so results are reproducible and explainable.
Content integrity	Every AI-generated CV, cover letter, and auto-filled answer must be traceable to a specific field in the user's verified profile — enforced by the grounding check in Section 15.5, not left to prompting alone.
Platform compliance	Automated form-filling and submission only operate within each job platform's documented API/integration terms (Section 16.2); the system never scrapes, impersonates a human to bypass bot detection, or submits outside a platform's permitted automation surface.
User control	The user must have complete control over automatic application behavior through global and per-job settings, including enabling/disabling auto-apply, setting minimum match scores, limiting daily applications, excluding employers or job categories, and revoking application permissions (FR-1.7–1.9, FR-7.10, FR-7.13, FR-7.14). Auto-submission is opt-in and reversible per job or globally; the user can always see, edit, and override anything the Agent generated before it is sent.
Consent management	Every automatic submission must be associated with a recorded user consent state and timestamp (`AuditLog` , Section 9.3). Revoking auto-apply permission must immediately prevent future automatic submissions — checked at the moment of submission, not only when the setting was last saved.

7. System Architecture

CareerForge uses a **single central backend** pattern: Next.js hosts the web frontend, the REST API, authentication, business logic, AI integration, and the portfolio engine. Both the browser client and the React Native (Expo) mobile app talk to this one backend over HTTPS/REST — there is no separate mobile-only backend to maintain, which keeps the system realistic to build and operate as a capstone.

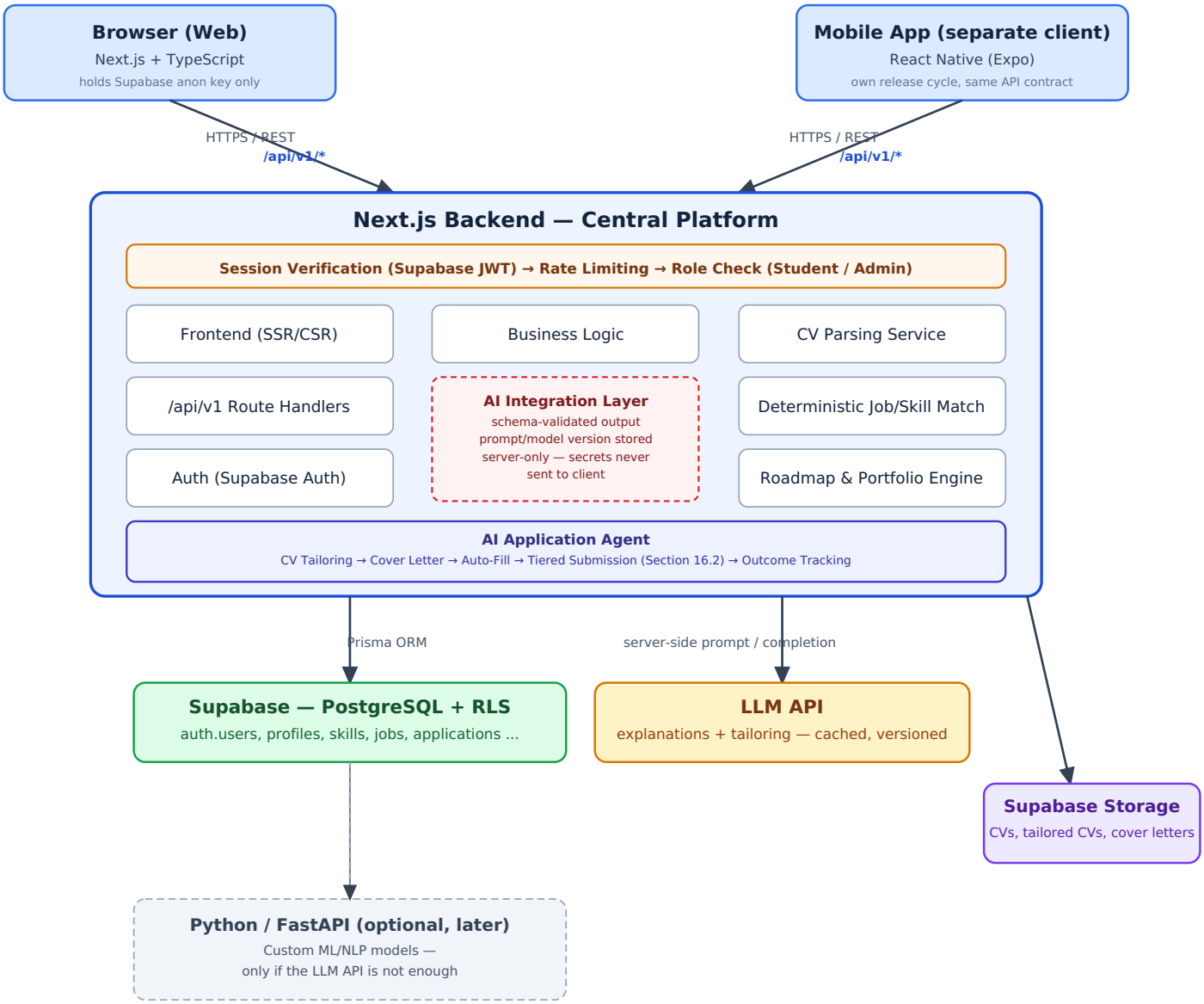


Figure 7.1 — Layered / central-platform architecture. Web and mobile are independent clients on one versioned `/api/v1` contract; the AI Integration Layer is the only place that ever holds the LLM API key or the Supabase service-role key. The Python microservice is an optional future extension, not a v1 dependency.

7.1 Key Architectural Decisions

- **One backend, two clients.** Next.js Route Handlers expose a single versioned REST API consumed identically by the Next.js web app and the React Native (Expo) mobile app. This avoids maintaining parallel business logic in two backend stacks.
- **AI called directly from Next.js (v1).** For the capstone scope, LLM calls are made server-side from Next.js API routes. A separate Python/FastAPI ML microservice is deliberately deferred until (if) custom NLP/ML models are actually required.
- **Prisma as the single ORM boundary.** One schema file is the source of truth for the entire data model (see Section 9), generating type-safe queries used across all API routes.
- **Supabase as the managed data platform.** Supabase provides the hosted PostgreSQL instance, Auth, and Storage as one managed service — Prisma connects to Supabase's Postgres connection string for typed queries/migrations, while **Row Level Security (RLS)** policies on tables such as `profiles`, `applications` and `cv` give a second, database-level guarantee that a user can only ever read/write their own data, independent of any bug in the API layer. Supabase Realtime is an optional, low-effort way to push live updates (e.g. roadmap progress, application status) to the client without building a custom WebSocket layer.
- **Explainability is architectural, not cosmetic.** Every AI-generated artifact (career match, gap analysis, CV feedback, roadmap) is persisted together with the profile snapshot that produced it, so results can be displayed with a "why" and re-computed on demand.
- **The AI Application Agent is a core component, not a bolt-on.** It sits inside the same AI Integration Layer as career matching and CV analysis (Section 15), reads only the verified profile and the deterministic match score M4 already computed, and is the only part of the system that writes to `Application`, `TailoredCV`, and `CoverLetter` (Section 9.3) — every write is behind the anti-fabrication check in Section 15.5 and the submission-tier routing in Section 16.2.

7.2 AI Application Agent — Internal Architecture

"AI Application Agent" is not one call to an LLM — it's a pipeline of nine internal components, each with a narrow, testable responsibility (Section 18.1 tests each independently):

Component	Responsibility
Job Eligibility Filter	First gate: checks the job against `ApplicationAgentSettings` — excluded companies/job types/locations (FR-7.14), daily submission limit (FR-7.13), and whether the user has already applied to this or an equivalent posting (FR-7.12, duplicate check). A job that fails this gate never reaches CV generation.
Deterministic Match Engine	The same scorer used by M4 (Section 15.3); re-run here so the Agent always acts on a current score, not a stale one from when the job was first discovered.
Application Policy Engine	Decides *whether* the Agent is allowed to act at all right now: is auto-apply enabled, is the score at/above the user's threshold (FR-1.8), has consent been revoked since the job was queued (Section 6, Consent management)? A "no" here routes to Tier 2/3 review instead of stopping silently.
CV Tailoring Service	Generates the job-specific CV from verified profile data only (FR-7.2).
Grounding Validation Service	The anti-fabrication gate (Section 15.5) — fact-diffs every generated CV/cover letter/answer against the verified profile and job posting before anything is persisted.
Application Form Mapper	Auto-fills fields it can confidently map from the profile; leaves anything ambiguous, subjective, or unavailable for the user (FR-7.4) rather than guessing.
Submission Router	Determines the permitted application workflow for each job: Tier 1 automatic submission when a supported API/integration exists *and* the user's policy permits automatic submission; otherwise Tier 2 assisted or Tier 3 manual preparation (Section 16.2).
Application Tracker	Persists the `Application` / `ApplicationAttempt` record and updates status over the application's lifecycle (FR-7.7).
Outcome Analysis Service	Computes the response/interview/offer metrics (FR-7.11) and feeds patterns back into M2/M4/M5 without touching the deterministic scoring algorithm itself (FR-7.8).

Two cross-cutting behaviors apply to every submission attempt, regardless of tier:

- **Failure handling.** If an automatic application fails because of an API error, invalid field, expired job, unavailable integration, or unexpected platform response, the Agent stops the submission process, records the failure reason on the `ApplicationAttempt` (Section 9.3), avoids repeated uncontrolled retries, and notifies the user or places the application into a review queue.
- **Idempotency.** Every submission request carries an idempotency key so that a network failure or a duplicated request can never accidentally submit the same application twice — the platform API (or the internal mock ATS, Section 16.2) is expected to de-duplicate on that key.

8. Technology Stack

Layer	Technology	Why
Web frontend	Next.js (React) + TypeScript	SSR/CSR in one framework; same language as the backend, so types are shared end-to-end.
Mobile app	React Native (Expo)	Single JS/TS codebase for iOS/Android, same language as the backend; talks to the same REST API as web.
Backend / API	Next.js Route Handlers & Server Actions	Removes the need for a second backend framework (e.g. Laravel) — one team, one language, one deploy.
Backend-as-a-Service	**Supabase**	One managed platform providing the hosted PostgreSQL database, Auth, Storage, and Realtime — removes most infrastructure work so the team can focus on product logic within a capstone timeline.
Database	PostgreSQL (hosted on Supabase)	Relational integrity for a highly relational domain (users ↔ skills ↔ courses ↔ jobs); strong JSON support for parsed CV data; Row Level Security enforced directly on the tables.
ORM	Prisma, pointed at the Supabase Postgres connection string	First-class TypeScript integration with Next.js; generates type-safe queries and migrations from one schema file while Supabase hosts and secures the actual database.
Authentication	Supabase Auth	Built-in email/password + OAuth (Google, GitHub, etc.), issues JWTs Next.js can verify on API routes, and integrates natively with Postgres RLS policies.
Styling	Tailwind CSS	Fast, consistent UI development for dashboards and forms.
AI / LLM	LLM API (e.g. Anthropic Claude API)	Explains deterministically-computed scores and generates language (CV/cover-letter text, roadmap notes) — never the score itself. Also powers the AI assistant (M9, stretch). No custom model training required.
File storage	Supabase Storage	S3-compatible object storage bundled with Supabase; stores uploaded CVs and portfolio images behind private buckets with signed URLs and the same RLS model.
Realtime *(optional)*	Supabase Realtime	Push live updates (application status changes, roadmap progress) to web/mobile clients without a custom WebSocket server.
Mobile ↔ backend	REST / JSON over HTTPS	Simple, well-understood contract shared by both clients; mobile can also use the Supabase JS SDK directly (via React Native) for auth/storage where convenient.
Deployment	Vercel (app) + Supabase (database, auth, storage)	Zero-friction CI/CD for Next.js; Supabase removes almost all database/auth/storage ops overhead for a capstone timeline.
Version control	Git / GitHub	Standard collaborative workflow, issue tracking, CI integration.
Optional, later	Python / FastAPI + ML models	Only introduced if a feature genuinely needs custom NLP/ML beyond what an LLM API call can do.

8.1 Why Supabase

Supabase was chosen over a self-managed PostgreSQL + Auth.js + S3 combination because it collapses three separate concerns (database hosting, authentication, file storage) into one service with one dashboard, one set of environment variables, and one security model (Postgres RLS) that covers both the API layer and the database layer. For a capstone with a fixed timeline, this materially reduces setup and operations work without giving up a real relational database or type-safe access via Prisma.

> **Example RLS policy** — a student can only ever see their own applications, enforced at the database layer regardless of what the API does:
>
> ```
> create policy "Users can view own applications"
> on applications for select
> using ( auth.uid() = user_id );
> ```

> **One thing deliberately avoided:** running Next.js and a fully separate mobile-facing backend as two parallel systems. Consolidating on a single Next.js backend — with Supabase as its managed data/auth/storage layer — is what keeps this capstone scoped to "one team can build and defend it" rather than "three backend stacks to maintain".

9. Database Design — Entity-Relationship Diagram

The schema is split conceptually into two domains that both hang off Supabase's own `auth.users` table: **(A) Profile & Academic data** — the raw facts about the student — and **(B) Career Intelligence data** — the AI-generated and market-facing artifacts derived from that profile. In Prisma, this is one schema file; it is shown as two diagrams here purely for readability.

9.1 Diagram A — Profile & Academic Domain

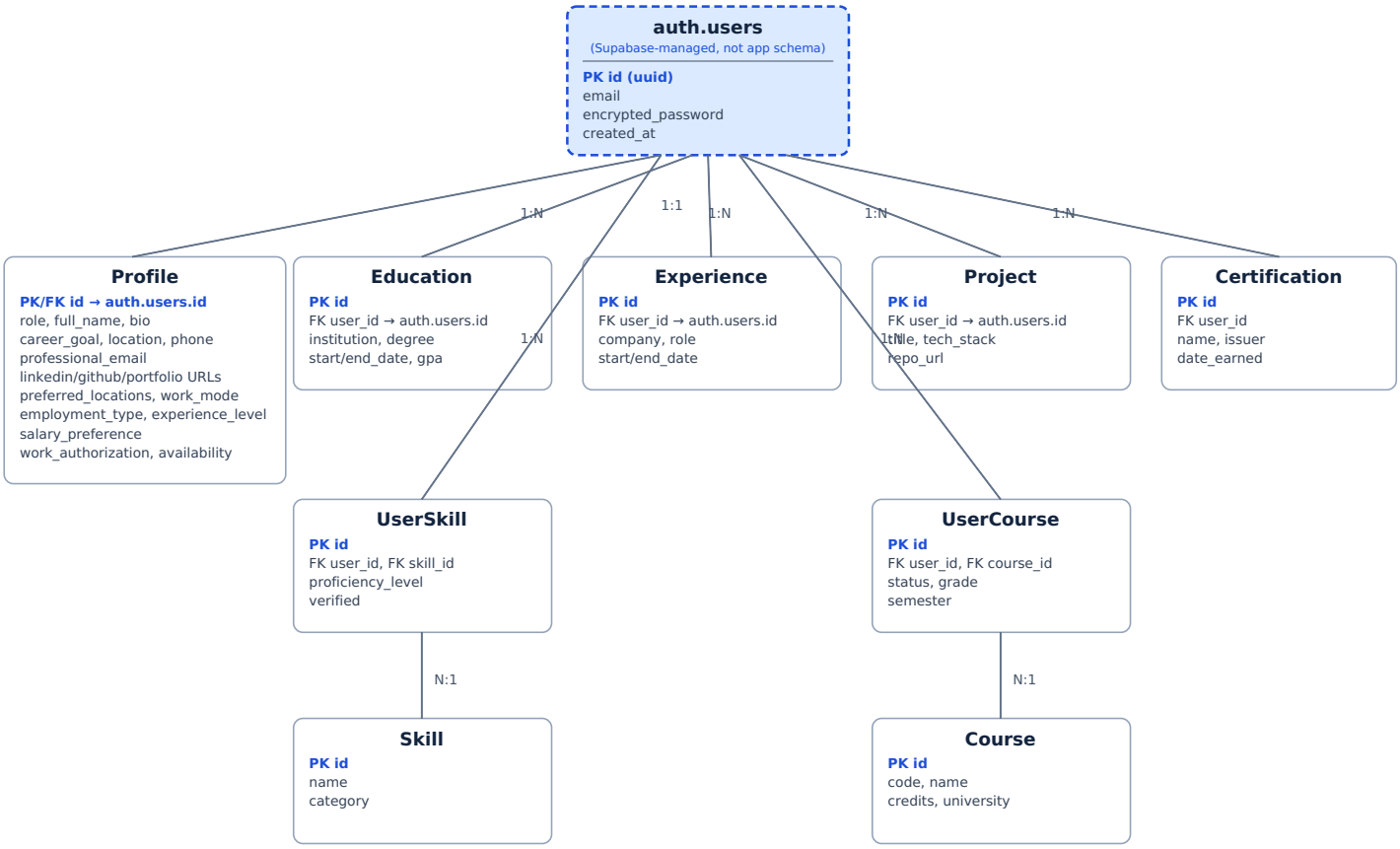


Figure 9.1 — Profile & Academic domain. `auth.users` is Supabase's own managed table (it stores credentials — the app never sees or stores a password hash); every app table's `user_id` / `id` is a foreign key straight into `auth.users.id`, and RLS policies compare that column against `auth.uid()`. UserSkill and UserCourse are join tables resolving the two many-to-many relationships (User↔Skill, User↔Course).

9.2 Diagram B — Career Intelligence Domain

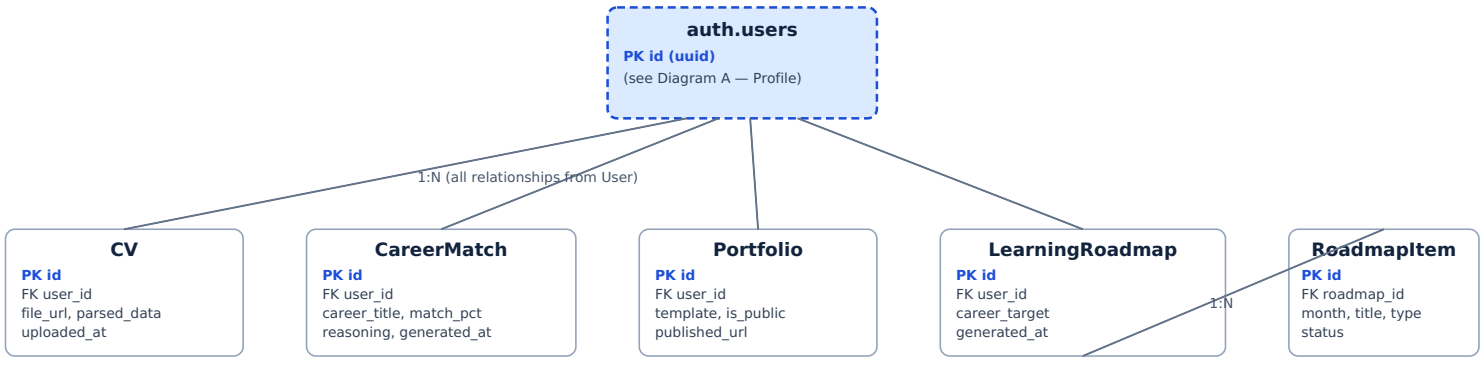


Figure 9.2 — Career Intelligence domain: the four AI-generated artifacts that aren't part of the application pipeline. CV/CareerMatch/Portfolio/LearningRoadmap are each owned by exactly one User and timestamped, satisfying the auditability NFR in Section 6. The AI Application Agent's own domain (Application, JobPosting, TailoredCV, CoverLetter, and its governance tables) is large enough to warrant its own diagram — see Figure 9.3.

9.3 Diagram C — AI Application Agent Domain

This is the data model behind M7 (Section 4, 7.2): everything the Agent reads, generates, and is governed by. It is deliberately the most detailed diagram in this report, because it is what makes the autonomy in Section 1 and Section 15.5 auditable rather than a black box.

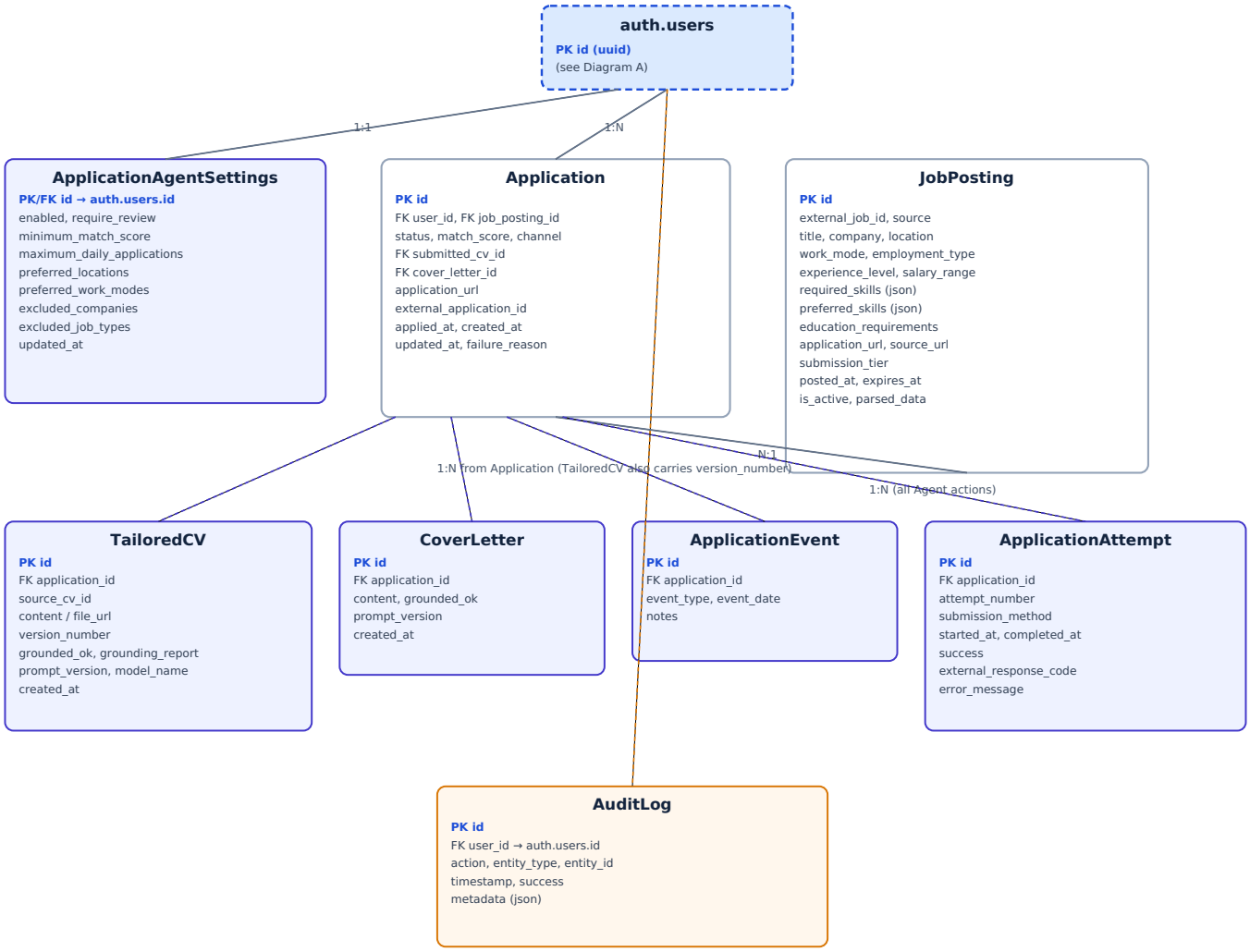


Figure 9.3 — AI Application Agent domain. `ApplicationAgentSettings` is the 1:1 configuration record the Job Eligibility Filter and Application Policy Engine (Section 7.2) check before anything is generated. `ApplicationAttempt` records every submission try (success or failure) separately from the `Application`'s current status, so a retried or failed Tier 1 submission is never silently lost. `AuditLog` captures every significant Agent action — CV generation, application preparation, submission, consent changes, and failed grounding checks — across this entire domain, addressing FR-A.5 and the auditability NFR (Section 6).

9.4 Database Constraints & Indexes

Beyond the primary/foreign keys already shown, these constraints and indexes are part of the schema design, not left as an implementation afterthought:

Table	Constraint / Index	Why
JobPosting	UNIQUE `(source, external_job_id)`	Prevents the same posting from a given source being ingested twice (Section 16.1).
JobPosting	INDEX on `(career_category, source, is_active)`	The hot path for M4's matching query and the eligibility filter's freshness check (FR-4.5).
Application	UNIQUE `(user_id, job_posting_id)` where `status != 'withdrawn'`	Database-level backstop for duplicate-application prevention (FR-7.12) — the Agent's own check (Section 7.2) is the primary defense, this is the guarantee if it's ever bypassed.
Application	INDEX on `(user_id, status)`	Supports the application tracker (UC-9) and outcome analysis (FR-7.11) without a full scan.
CareerMatch, Application, TailoredCV, CoverLetter, ApplicationEvent, ApplicationAttempt, AuditLog	`created_at` / `timestamp` NOT NULL, default `now()`	Every AI-generated or Agent-driven row is timestamped — required by the auditability NFR (Section 6).
CareerMatch	CHECK `match_pct BETWEEN 0 AND 100`	A percentage field that can't silently become -5 or 140 through a scoring bug.
ApplicationAgentSettings	CHECK `minimum_match_score BETWEEN 0 AND 100` ; CHECK `maximum_daily_applications > 0`	Enforces FR-7.13 at the data layer, not just in application code.
Education, Experience, Project, Certification, UserSkill, UserCourse	FK `user_id` → `auth.users.id` ON DELETE CASCADE	Deleting a user account (right-to-erasure) cleans up owned rows automatically rather than leaving orphans.
TailoredCV, CoverLetter,	FK `application_id` ON DELETE	These rows have no meaning without their parent `Application` .

ApplicationEvent, ApplicationAttempt	CASCADE	
AuditLog	FK `user_id` ON DELETE SET NULL (not CASCADE)	Audit history is retained for compliance even if the account is later deleted.

10. Use Case Model

Three actors interact with CareerForge: the **Student** (primary actor, on web or mobile), the **Admin** (curates job postings and taxonomy data), and the **LLM API**, an external system invoked by several use cases rather than driven by a human.

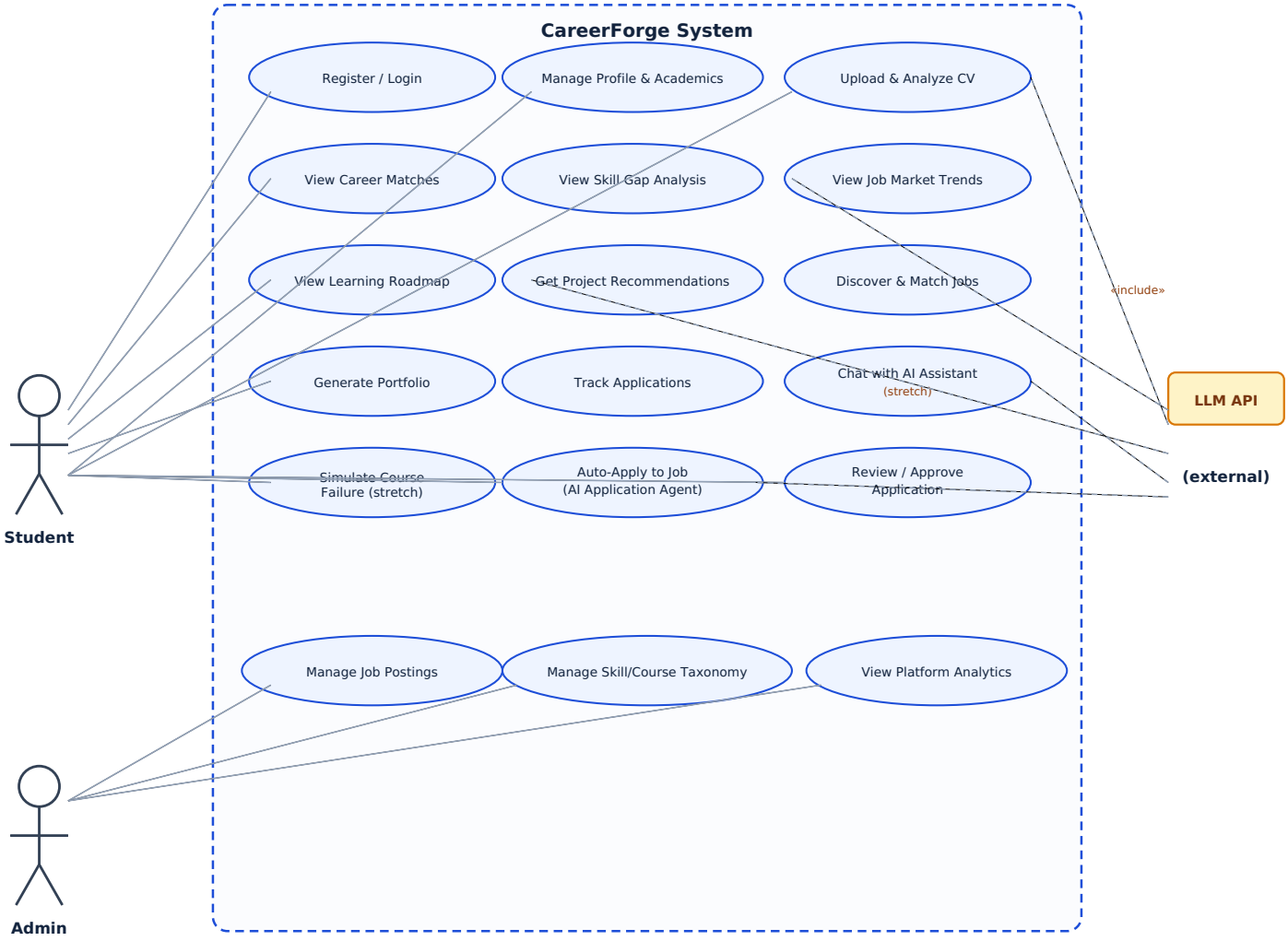


Figure 10.1 — Use case diagram. Dashed «include» lines show which student-facing use cases delegate reasoning to the LLM API (career matching, CV analysis, project recommendations, CV/cover-letter tailoring, and the AI assistant).

10.1 Key Use Case Descriptions

Use Case	Primary Actor	Description & Trigger
UC-1 Upload & Analyze CV	Student	Student uploads a baseline CV file and selects a target job/career. System parses the CV, computes a deterministic CV-to-job compatibility score (FR-6.2), and calls the LLM to explain the score and generate section-level feedback. Precondition: profile exists. Postcondition: a CV analysis record is stored.
UC-2 View Career Matches	Student	Student requests career suggestions. System computes deterministic career-match scores from the current profile snapshot (FR-2.1), sends the scores and profile to the LLM for a plain-language explanation, and persists both the scores and the reasoning.
UC-3 View Skill Gap Analysis	Student	Student selects a target career. System compares the profile's skills against that career's requirement set and job-market aggregate data, returning a Critical/Important/Optional breakdown.
UC-4 Discover & Match Jobs	Student	Student browses job postings; system computes a compatibility score per posting and an Apply / Not yet recommendation.
UC-5 Generate Portfolio	Student	Student selects a template. System composes a public portfolio page directly from existing profile data — no re-entry of information.
UC-6 Manage Job Postings	Admin	Admin creates/edits/removes job postings and their parsed required-skill lists that feed UC-3 and UC-4.
UC-7 Auto-Apply to Job	Student (AI Application Agent)	For a job at/above the student's match threshold, the Job Eligibility Filter and Application Policy Engine (Section 7.2) first check duplicate-application status, daily limits, and exclusions (FR-7.12–7.14); if it passes, the system tailors a CV and (if required) a cover letter from verified profile data only, auto-fills the platform's form, and either submits automatically (Tier 1 platform, auto-submit enabled) or stages the application for one-click confirmation (Tier 2/3, Section 16.2). Precondition: eligibility/policy checks and the grounding check (Section 15.5) pass. Postcondition: an `Application` record with its

		`TailoredCV` / `CoverLetter` and a logged `ApplicationAttempt` is stored.
UC-8 Review / Approve Application	Student	Student inspects a system-prepared application (tailored CV, cover letter, filled answers) before it is sent, edits anything, then confirms or declines. Required for every Tier 2/3 platform and optional for Tier 1 if the student has not enabled silent auto-submit.
UC-9 Track Applications	Student	Student views all applications with current status; updates status manually or receives it from platform polling/webhooks where available (FR-7.7).
UC-10 Manage Skill/Course Taxonomy	Admin	Admin creates, edits, activates, or deactivates skills, career categories, courses, and related taxonomy information used by the matching and recommendation engines (FR-A.2).
UC-11 View Platform Analytics	Admin	Admin views aggregated statistics about users, jobs, applications, interviews, offers, and rejections (FR-A.4).
UC-12 Configure Application Sources	Admin	Admin configures supported job sources, ATS integrations, submission tiers, and integration status used by the AI Application Agent (FR-A.3).

11. Sequence Diagrams

The three flows below cover the highest-value interactions: CV analysis, career matching, and job matching with application tracking. All three follow the same pattern — client calls the Next.js API, the API reads/writes Supabase (Postgres/Storage), and calls the LLM API only where reasoning (not retrieval) is required.

11.1 CV Upload & Analysis

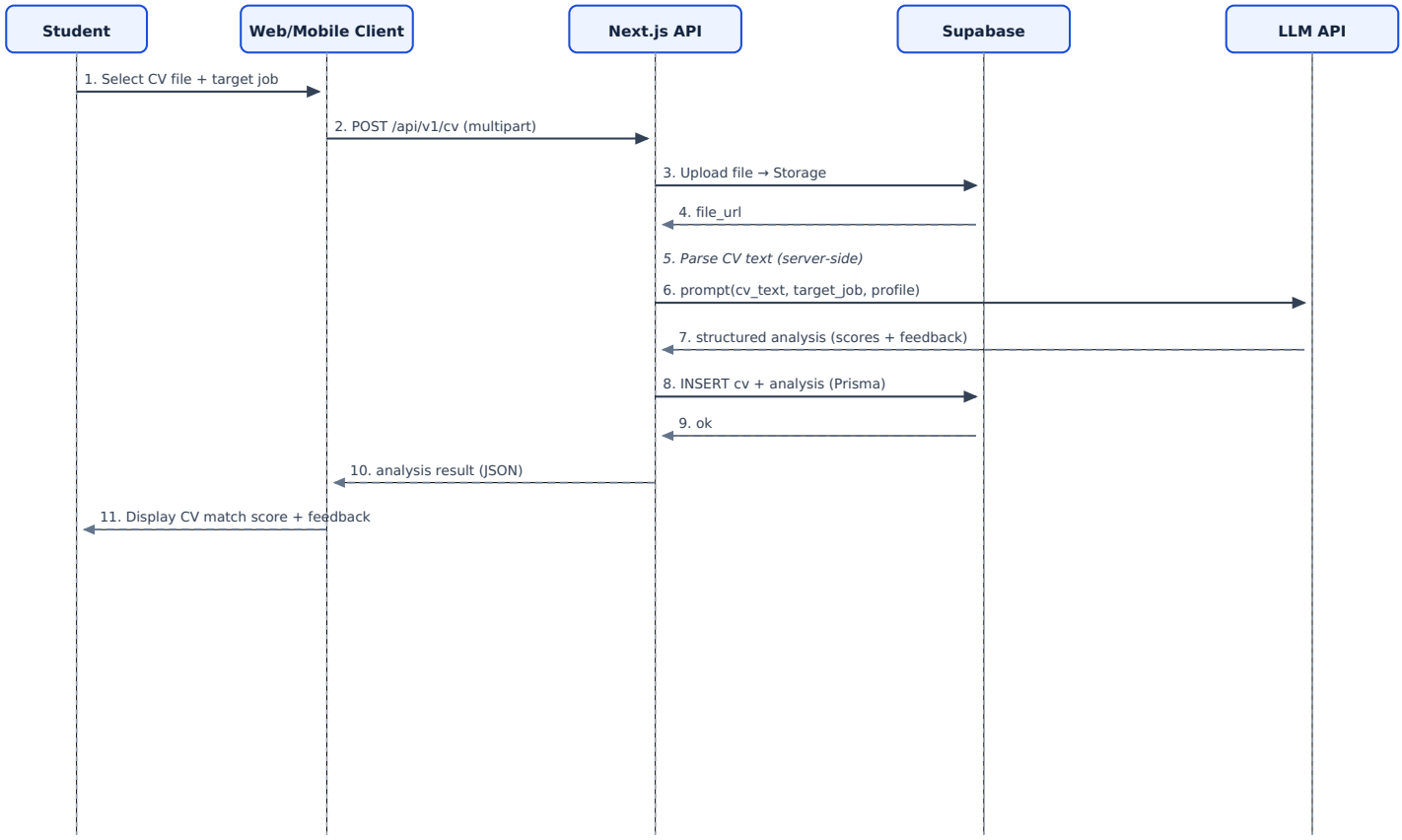


Figure 11.1 — CV upload and analysis. Parsing happens server-side before the LLM call so the prompt contains clean, structured text rather than raw file bytes.

11.2 Career Match Generation

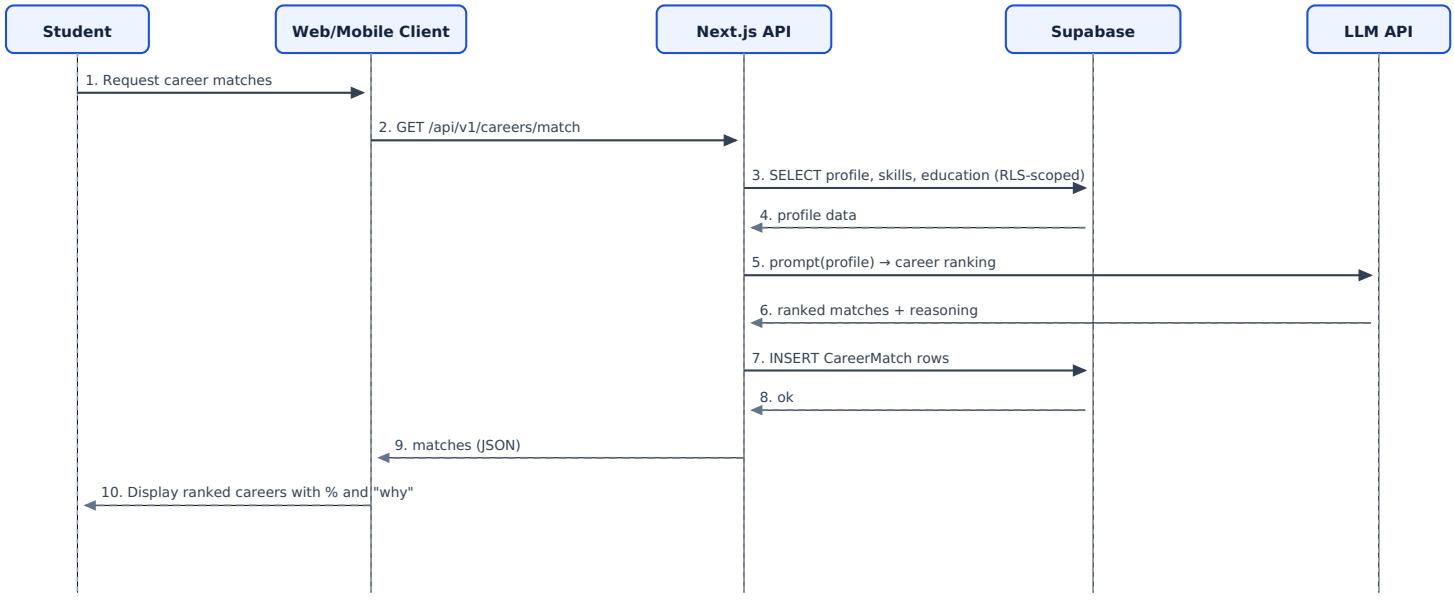


Figure 11.2 — Career match generation. Every match is written back to Postgres so it can be displayed, audited, and re-used by the skill-gap and roadmap modules without re-calling the LLM.

11.3 Job Matching & Application Tracking

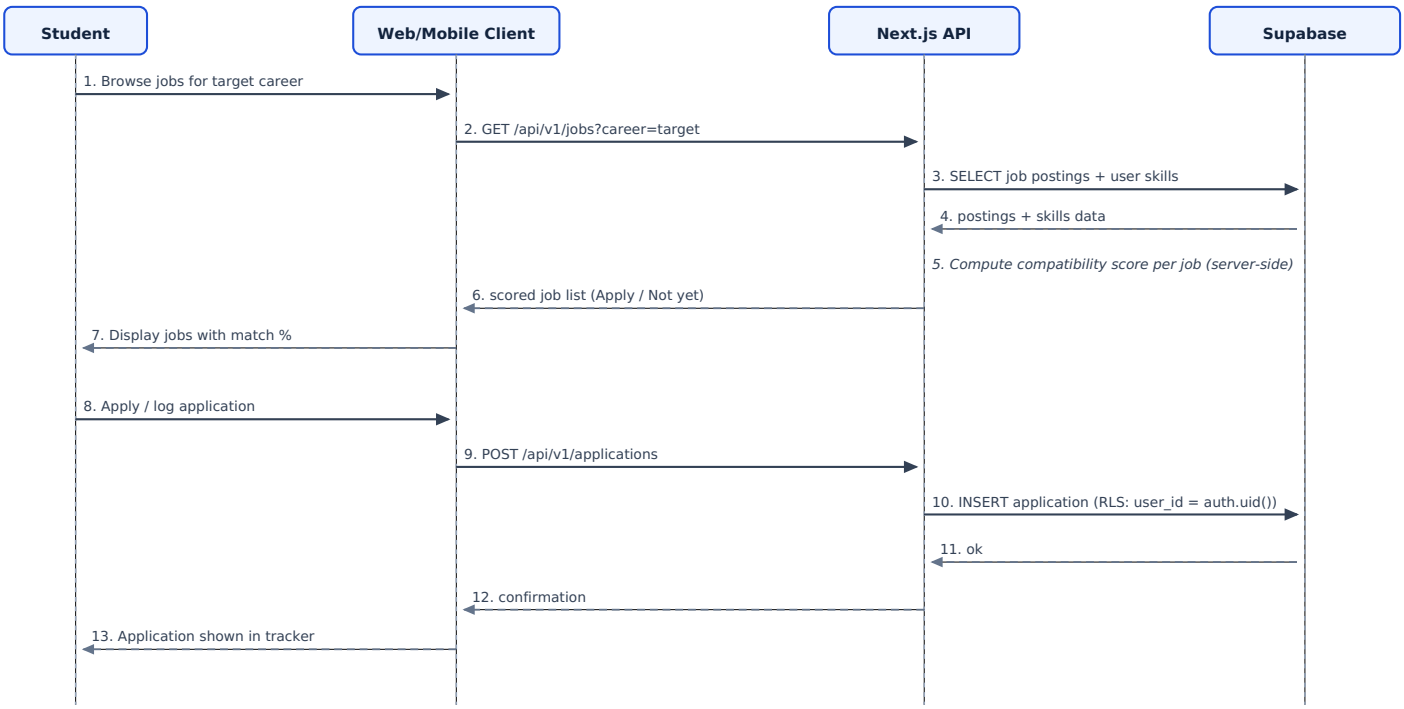


Figure 11.3 — Job matching and application logging. Row Level Security on the `applications` table means step 10 fails at the database layer if the API ever tried to insert a row for a different user_id.

11.4 AI Application Agent — Discover, Tailor, Apply, Track

This is the detailed version of the pipeline in Section 12: what happens, end to end, for one job the Agent decides is worth acting on. It replaces "log an application" (Figure 11.3, step 8) with full CV/cover-letter generation, grounding validation, auto-fill, and tiered submission.

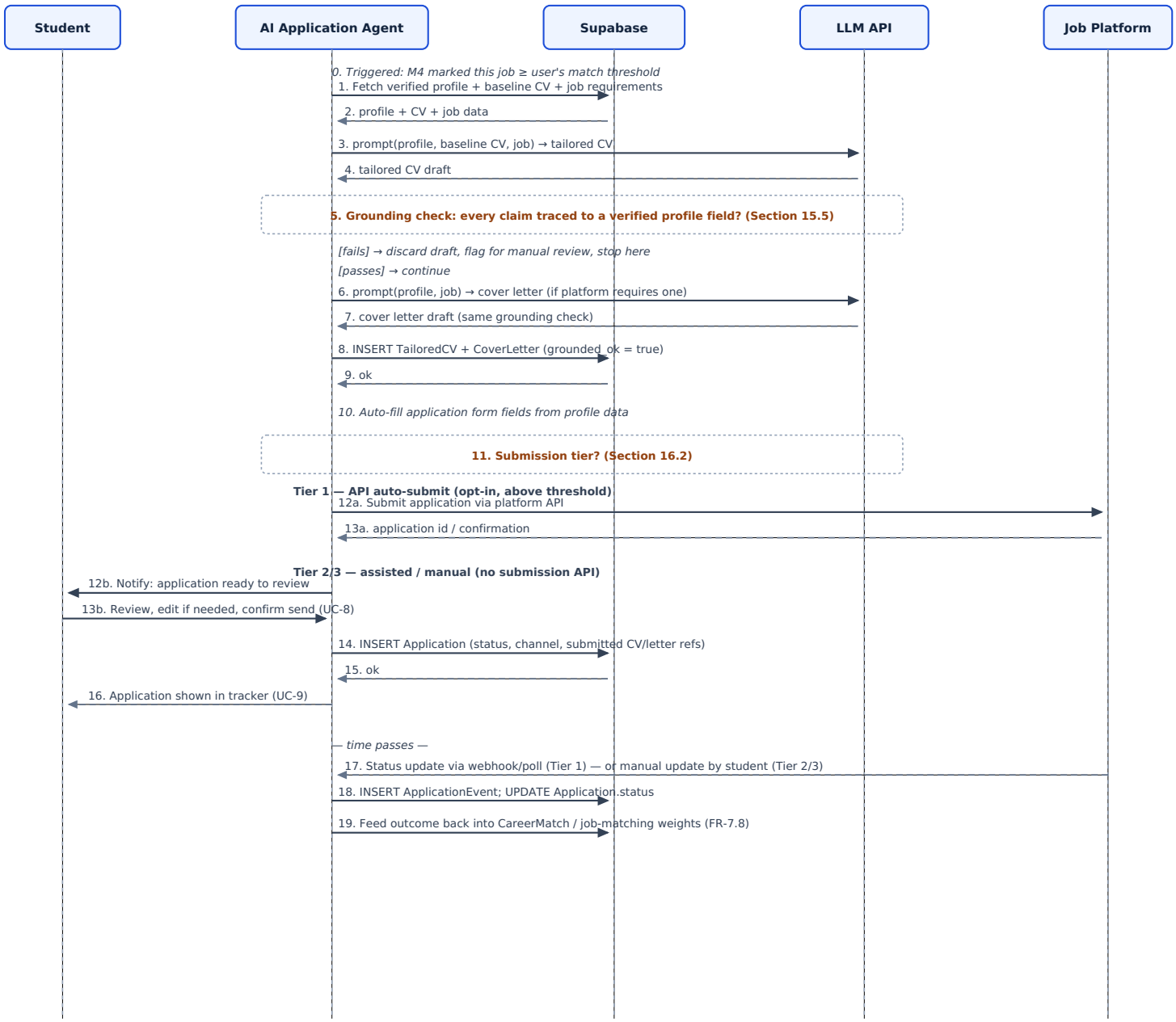


Figure 11.4 — The AI Application Agent pipeline (Section 12). Step 5's grounding check is a hard gate, not a suggestion — a tailored CV or cover letter that fails it is never persisted with `grounded_ok = true` and can never reach step 14. Tier 1 vs. Tier 2/3 routing (step 11) is decided per job platform, not per user (Section 16.2).

12. System Data Flow

At the highest level, everything the platform does is one pipeline: turn a student's raw profile into an explainable, continuously-updated action plan — and, for the AI Application Agent specifically, into real submitted applications.

12.1 The CareerForge Primary Workflow

This is the complete, end-to-end workflow — from a raw profile to an improved recommendation — with every gate the AI Application Agent must pass through made explicit. Figure 11.4 diagrams the sequence behind stages 6–13 in detail.

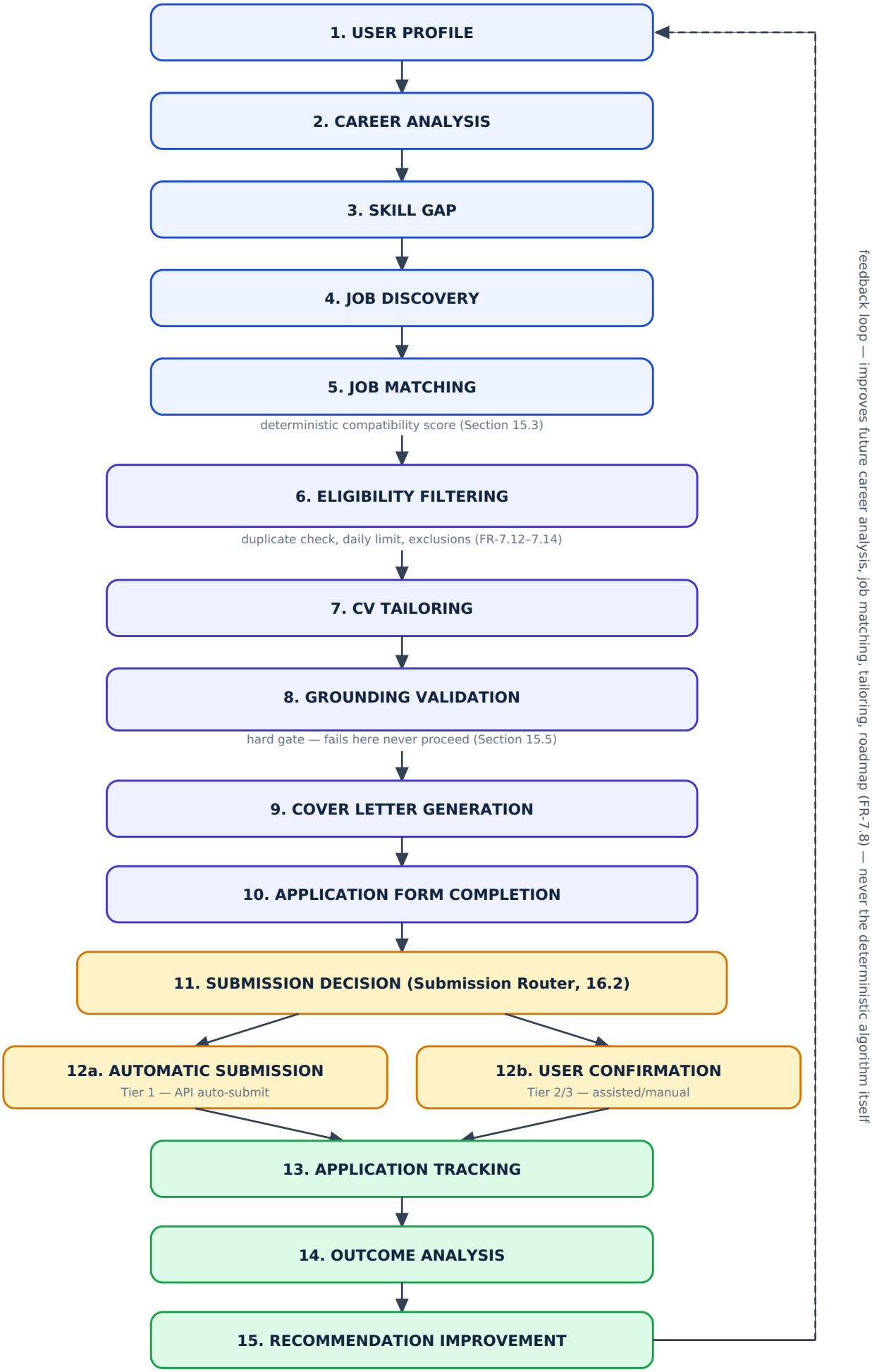


Figure 12.1 — CareerForge's primary workflow: User Profile → Career Analysis → Skill Gap → Job Discovery → Job Matching → Eligibility Filtering → CV Tailoring → Grounding Validation → Cover Letter Generation → Application Form Completion → Submission Decision → Automatic Submission or User Confirmation →

Application Tracking → Outcome Analysis → Recommendation Improvement. Stages 6–10 (indigo) are internal to the AI Application Agent (Section 7.2); stages 11–12 (amber) are the Submission Router's tiered decision (Section 16.2).

12.2 Whole-Platform Data Flow

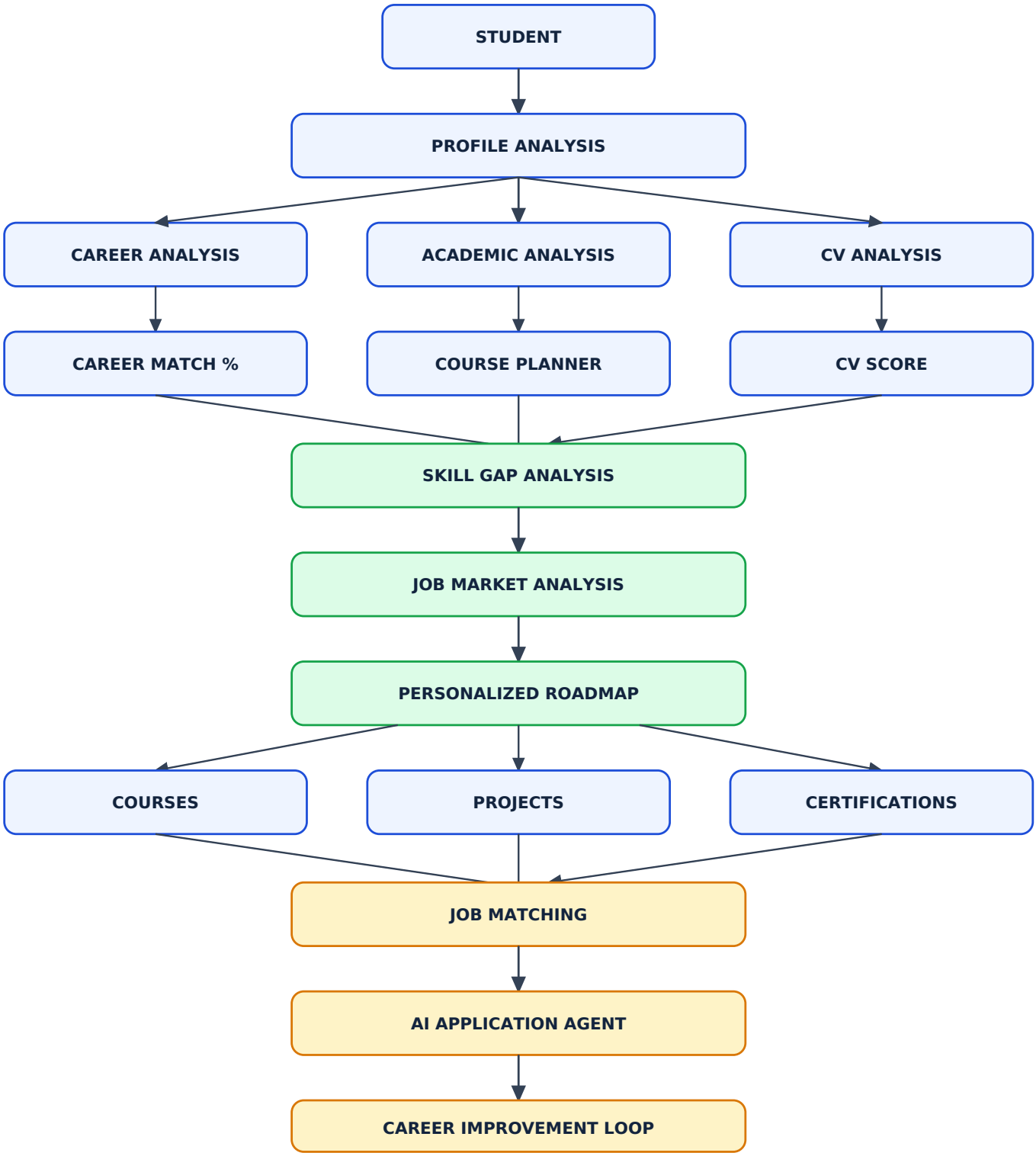


*Figure 12.2 — End-to-end data flow. The bottom loop (AI Application Agent → Career Improvement) feeds outcomes back into Career/Skill Analysis, which is what makes the platform **closed-loop** rather than a one-shot recommender. See Figure 12.1 for the Agent's own internal pipeline.*

13. API Design

All endpoints are Next.js Route Handlers under the explicit, versioned prefix `/api/v1` — not a bare `/api` implicitly treated as v1. This gives the API a real contract: a breaking change ships as `/api/v2` alongside the old routes, so the web app and the independently-released React Native (Expo) app are never forced to deploy in lockstep. Every request (except register/login) must include a Supabase session JWT in the `Authorization: Bearer <token>` header; the API verifies it, applies per-user rate limiting (Section 13.1), and Postgres RLS provides a second, independent access check at the database layer.

Method	Endpoint	Purpose	Module
POST	`/api/v1/auth/register`	Create account (proxies to Supabase Auth).	M1
POST	`/api/v1/auth/login`	Authenticate, return session.	M1
GET/PUT	`/api/v1/profile`	Read/update the current user's profile.	M1
GET/POST/DELETE	`/api/v1/skills`	List/add/remove skills on the profile.	M1
GET/POST/DELETE	`/api/v1/education`	Manage education history entries.	M1
GET/POST/DELETE	`/api/v1/experience`	Manage work-experience entries.	M1
GET/POST/DELETE	`/api/v1/projects`	Manage personal/academic projects.	M1
GET	`/api/v1/careers/match`	Return ranked career matches (deterministic score, FR-2.1) with an LLM-generated explanation.	M2
GET	`/api/v1/careers/:id/gap`	Skill-gap breakdown for a target career.	M2
GET/POST	`/api/v1/courses`	List available courses / enroll a user course.	M3
GET	`/api/v1/courses/plan`	Degree progress + recommended next courses.	M3
GET	`/api/v1/jobs`	Browse job postings with computed match scores.	M4
GET	`/api/v1/jobs/market`	Aggregated skill-demand stats across postings.	M4
GET	`/api/v1/roadmap`	Fetch/generate the personalized learning roadmap.	M5
PATCH	`/api/v1/roadmap/:itemId`	Mark a roadmap item complete; triggers re-adaptation.	M5
GET	`/api/v1/projects/recommendations`	LLM-generated project ideas targeting current gaps.	M5
POST	`/api/v1/cv`	Upload CV; parse, compute deterministic match score, and request an LLM explanation + improvement suggestions (FR-6.2).	M6
GET/PUT	`/api/v1/portfolio`	Fetch/regenerate the public portfolio page.	M6
GET/POST	`/api/v1/applications`	List applications / manually create one.	M7
POST	`/api/v1/applications/:jobId/tailor`	Generate (or re-generate) a grounded `TailoredCV` + `CoverLetter` for a job (Section 15.5).	M7
POST	`/api/v1/applications/:jobId/autofill`	Auto-fill available application-form fields from profile data without submitting.	M7
POST	`/api/v1/applications/:id/submit`	Submit via Tier 1 platform API, or mark a Tier 2/3 application as sent after user confirmation (Section 16.2).	M7
PATCH	`/api/v1/applications/:id/status`	Update application status; also the target of Tier 1 platform status webhooks (FR-7.7).	M7
GET/PUT	`/api/v1/profile/preferences`	View/update job-application preferences (location, work mode, employment type, salary, exclusions) — FR-1.7, FR-1.9.	M1
GET/PATCH	`/api/v1/settings/application-agent`	View/update `ApplicationAgentSettings`: auto-apply enable, minimum match score, daily limit, excluded companies/job types (FR-1.8, FR-7.10, FR-7.13, FR-7.14).	M7
POST	`/api/v1/assistant`	Chat with the profile-aware AI assistant. **STRETCH** — not part of the mandatory v1 implementation checklist.	M9
POST/PUT/DELETE	`/api/v1/admin/jobs`	Admin: create, edit, deactivate, delete job postings (FR-A.1).	Admin
POST/PUT/DELETE	`/api/v1/admin/taxonomy`	Admin: manage skills, career categories, course taxonomy, experience levels (FR-A.2).	Admin
GET/PUT	`/api/v1/admin/sources`	Admin: configure supported job sources/ATS integrations and their submission tier (FR-A.3, UC-12).	Admin
GET	`/api/v1/admin/analytics`	Admin: platform statistics — users, jobs, applications, interviews, offers, rejections (FR-A.4).	Admin
GET	`/api/v1/admin/audit-log`	Admin: view the recorded `AuditLog` of significant Agent and admin actions (FR-A.5).	Admin

13.1 Rate Limiting & Quotas

Every route is rate-limited by user id (falling back to IP for unauthenticated routes) using a sliding-window limiter (e.g. Upstash Ratelimit backed by Redis, or a Postgres-table fallback for a zero-extra-infra v1). AI-backed routes get tighter, separate quotas since they carry a real per-call cost:

Route class	Example limit	Reason
Standard CRUD (`/api/v1/profile` , `/api/v1/skills` , ...)	~60 req/min/user	Protect the database from accidental client loops.
AI-backed (`/api/v1/careers/match` , `/api/v1/cv`)	~10 req/hour/user	Bound LLM API spend per user; paired with caching (Section 15.4). `/api/v1/assistant` (M9, stretch) would follow the same limit if/when built.
Auth (`/api/v1/auth/*`)	~5 req/min/IP	Slow down credential-stuffing/brute-force attempts.

A request over quota returns `429 Too Many Requests` with a `Retry-After` header; both clients (web and React Native) surface this as a friendly "try again in a moment" state rather than a generic error.

14. Security, Session & Secrets Management

This section makes explicit what Section 6 (NFRs) and Section 7 (Architecture) already assume, because it is easy to get subtly wrong: who holds which credential, and how a session is actually verified on every request.

14.1 Session & Authentication Handling

- Supabase Auth issues a short-lived access token (JWT) plus a refresh token on login. The web app stores these in **httpOnly, secure, SameSite=Lax cookies** set by a Next.js server action — never in `localStorage`, which is readable by any injected script (XSS exposure).
- The React Native app stores tokens in **secure device storage** (Expo SecureStore / Keychain / Keystore), not `AsyncStorage`, which is unencrypted.
- Every `/api/v1/*` route (except `/auth/register` and `/auth/login`) runs through shared middleware that verifies the access token's signature and expiry, refreshes it transparently when close to expiry, and rejects the request with `401` otherwise.
- Refresh tokens are rotated on use and revocable server-side (sign-out-everywhere), so a leaked refresh token has a bounded lifetime rather than being a permanent credential.

14.2 Secrets Stay Server-Side

> **Rule:** the LLM API key and the Supabase *service role* key are read only inside Next.js server code (Route Handlers, server actions) via server-only environment variables (no `NEXT_PUBLIC_` prefix). The browser and the React Native app hold only the Supabase **anon** key, which is safe to ship publicly precisely because RLS policies — not the key itself — are what restrict data access.

Secret	Held by	Notes
Supabase anon key	Web + Mobile (client)	Public by design; safe only because every table has RLS enabled.
Supabase service role key	Server only	Bypasses RLS — used solely for admin-style server jobs (e.g. seeding job postings), never in a request path a client can trigger directly.
LLM API key	Server only	Never referenced in any client bundle; calls happen inside the AI Integration Layer (Section 7).
Database direct connection string	Server / CI only	Used by Prisma for migrations; not reachable from either client.

14.3 Role-Based Authorization

Authorization is enforced twice, independently: the API middleware checks the caller's `profiles.role` (student/admin) before executing admin-only route handlers (`/api/v1/admin/*`), and Postgres RLS policies additionally scope every query by `auth.uid()` and, where relevant, role — so a bug in one layer doesn't expose the other.

14.4 Third-Party Platform Credentials (Tier 1 Integrations)

Where a Tier 1 job platform/ATS (Section 16.2) requires OAuth to submit on the user's behalf, the resulting access/refresh tokens are encrypted at rest (Supabase Vault or application-level envelope encryption) and scoped to exactly the application-submission permission the integration needs — never a broader account-management scope. Tokens are never logged, never returned to the client, and are usable only by the AI Application Agent's server-side submission step (Figure 11.4, step 12a); revoking consent (FR-7.10) deletes the stored token immediately.

15. AI Integration Reliability

Calling an LLM is the one part of the system that is non-deterministic and can fail in new ways (malformed output, hallucinated fields, drifting quality across model updates). CareerForge treats this as an engineering problem, not an implementation detail.

15.0 Deterministic Logic vs. AI/LLM: What Each Layer Does

To be explicit about what "AI" means in this project — and to avoid CareerForge being read as "a normal application with a chatbot added" — every computation in the system falls into exactly one of two layers. Nothing in the deterministic column is ever delegated to the LLM, and nothing in the AI column is a pretense for logic that should have been deterministic.

Deterministic system (plain code, testable, reproducible)	**AI / LLM (language generation only)**
Career-match score (FR-2.1)	Explain the career-match score in plain language
Job compatibility score (FR-4.2)	Explain missing-skill/missing-experience reasoning per job
Skill-gap classification: Critical/Important/Optional (FR-2.3)	Generate CV wording tailored to a specific job (FR-7.2)
CV-to-job compatibility score (FR-6.2)	Generate cover letters (FR-7.3)
Degree progress calculation (FR-3.2)	Generate project recommendations and their rationale (Section 4, M5)
Eligibility rules: threshold, exclusions (FR-7.13, FR-7.14)	Generate learning-roadmap explanations
Duplicate-application detection (FR-7.12)	Profile-aware assistant answers (M9, stretch)
Application daily-limit enforcement (FR-7.13)	
Submission-tier routing (Section 16.2)	

This table is the ground truth for Section 5's requirements and Section 7.2's architecture — if a future change makes the LLM responsible for anything in the left column, that is a design regression, not a refactor.

15.1 Output Validation

Every AI-backed route defines a strict response schema (e.g. a Zod schema) that the raw LLM output must parse against before it is trusted. On a validation failure, the system retries once with a stricter "return valid JSON only" instruction; if it still fails, the route returns a clear "couldn't generate this right now" error rather than persisting or displaying malformed data.

15.2 Prompt & Model Versioning

Every AI-generated record (`CareerMatch`, CV analysis, roadmap) stores the `prompt_version` and `model_name` that produced it, alongside the profile snapshot (Section 7.1). This makes results reproducible, lets the team A/B a prompt change safely, and explains why two matches for the same user might differ if generated weeks apart under different prompt versions.

15.3 Deterministic Matching, AI Explanation

> **Important distinction:** compatibility scores (career match %, job match %, skill-gap severity) are computed by a **deterministic, rule-based scoring function** in the backend (weighted skill overlap, experience-level checks, etc.) — not by asking the LLM to "give a percentage". The LLM is called *only* to generate the plain-language explanation for a score the system already computed. This keeps scores stable, auditable, and cheap to recompute, while still getting natural-language "why" text.

15.4 Caching & Cost Control

- A `CareerMatch` (or CV analysis) is served from its stored record and is only re-generated when the underlying profile has materially changed (a content hash of the relevant profile fields is compared) or the user explicitly requests a refresh.
- Identical prompts within a short window (e.g. re-opening the same CV analysis) are served from a short-lived cache keyed by the content hash + prompt version, avoiding duplicate LLM spend.
- Combined with the AI-route rate limits in Section 13.1, this bounds per-user and total LLM cost to a predictable ceiling.

15.5 Anti-Fabrication Guarantee (AI Application Agent)

> **Hard requirement:** the AI Application Agent must enforce factual grounding for all applicant-related claims. The Agent may rewrite, summarize, reorder, and prioritize verified information, but it may not create new applicant facts. Job-specific information — company name, job title, job requirements — may be taken from the corresponding verified job posting (`JobPosting`, Section 9.3), since that is not a claim about the applicant. This is enforced as a pipeline gate (Figure 11.4, step 5; Figure 12.1, stage 8), not left to prompt wording alone.

The grounding check works in three layers:

1. **Constrained generation.** The prompt supplies the LLM with the user's structured profile fields (skills, education, experience, projects, certifications) as its sole source of applicant facts — plus the job posting as the sole source of job facts — with an explicit instruction to

rephrase and re-prioritize but never invent, and to omit rather than guess when a job asks for something the profile doesn't contain.

2. **Automated fact-diff.** Before a generated CV/cover letter is persisted, a deterministic post-processing step extracts factual claims about the applicant (skills named, employers, dates, degree titles, project names, metrics, achievements) from the output and checks each one against the verified profile record it was generated from. Any claim that doesn't match sets `grounded_ok = false` and the content is discarded and flagged for manual review — never silently corrected or reworded by another LLM call, which would just relocate the risk.
3. **No autonomous profile edits.** The Agent can only *read* the profile; it has no code path that writes to `Profile`, `Education`, `Experience`, `Project`, `Certification`, or `UserSkill` (Section 9.1) — so it cannot "pad" the source data to make its own later output pass the grounding check.

> **Uncertain or sensitive fields are never inferred.** If a generated application answer cannot be confidently grounded in verified user data, the Agent must mark the field as requiring user input and must not submit an inferred answer (FR-7.4). This applies particularly to legal, work-authorization, sponsorship, demographic, salary, relocation, availability, and other sensitive application questions — exactly the fields where a plausible-sounding guess would be the most harmful kind of fabrication.

A generated document that fails grounding is a bug to be reviewed, not a job that was skipped — failed-grounding events are logged (Section 18.5, `AuditLog`) so patterns (e.g. one job platform's listings triggering more failures) can be investigated.

16. Job-Market Data & Application Submission Channels

16.1 Job-Market Data Sourcing

The `JobPosting` table (Section 9.3) needs to be filled from somewhere, and this choice has real licensing and reliability implications, so it is called out explicitly rather than left implicit.

Approach	Description	Used in
Seeded / admin-curated dataset	Admin (Section 10, UC-6) manually adds and edits postings through `/api/v1/admin/jobs`. For the capstone demonstration, CareerForge will initially use this admin-curated (or otherwise legally permitted) job dataset to guarantee reliable and reproducible demonstrations — zero legal/ToS risk and fully under the team's control.	v1 (default)
Public job-board API	Ingest postings from an API with an explicit developer program (e.g. Adzuna, Jooble, USAJobs) via a scheduled job that maps their fields onto `JobPosting`. May be integrated later, once that provider's terms, rate limits, and access requirements have been verified — not assumed available at v1.	Future enhancement
Scraping	Explicitly out of scope — scraping sites like LinkedIn or Indeed typically violates their Terms of Service and is a legal/ethical risk not worth taking for a capstone project.	Not used

Whichever source is used, postings are normalized into the same shape (Section 9.3's `JobPosting` fields) before insertion, so the matching engine (Section 15.3) never needs to know where a posting came from. Each posting stores its **source provider, external job identifier, retrieval timestamp, application URL, and submission tier** — this is what lets the system determine whether a posting is still current and which application workflow is permitted (FR-4.1, FR-A.3).

> **Job freshness:** the system must identify stale or expired job postings — past their retrieval freshness window or past `expires_at` — and exclude them from active matching (FR-4.5). The AI Application Agent's eligibility filter (Section 7.2) re-checks `is_active` immediately before acting, since a posting can go stale between discovery and the Agent processing it.

16.2 Application Submission Tiers

"Automatically applying" means something different on every platform, so the Agent routes each job through one of three tiers rather than assuming a single behavior everywhere (Figure 11.4, step 11; Figure 12.1, stage 11):

Tier	Channel	Behavior & Examples
1	**API auto-submit**	The employer/platform exposes an official application-submission API or ATS integration (e.g. Greenhouse, Lever, SmartRecruiters, Workday job-board APIs). The Agent submits directly, but only when the user has opted in to auto-submit — globally or for that job — and the job's match score is at or above the user's chosen threshold (FR-7.5).
2	**Assisted apply**	No submission API, but the platform doesn't prohibit a human using a pre-filled form. The Agent tailors the CV/cover letter and auto-fills every field it can, then **stops and requires one explicit user action** (UC-8) to actually send it — the Agent never simulates clicks or submits on the user's behalf here.
3	**Manual bundle**	Application is by email or a platform with no safe automation surface at all. The Agent prepares the tailored CV, cover letter, and a suggested email draft as a downloadable/attachable bundle for the user to send themselves.

> **Why not automate everything:** several major job platforms' Terms of Service explicitly prohibit automated form submission or bot-driven "Easy Apply"-style actions. Treating Tier 1 as the only safe form of automation — and defaulting every other platform to Tier 2/3 — is a deliberate design choice, not a missing feature. CareerForge is designed to automate the job-application process end-to-end where the target platform provides a permitted and supported application API/integration; on platforms without one, it prepares everything but requires one explicit user confirmation. The system never bypasses CAPTCHA, bot detection, authentication controls, or platform restrictions to force Tier 1 behavior where it isn't supported. The system determines a job's tier from the platform/ATS the posting came from (admin-configured per source, Section 16.1, FR-A.3), never by attempting an integration and falling back only on failure.

For the capstone build, this means **one controlled Tier 1 integration** — preferably against a documented test/sandbox environment where the real ATS provides one — rather than promising breadth across multiple platforms. Additional ATS integrations remain a future enhancement (Section 21), not a v1 deliverable.

> **Mock ATS for demonstration:** to demonstrate the complete autonomous workflow safely without depending on a production job platform's cooperation, CareerForge provides a controlled mock ATS environment that reproduces an application form and a submission API. This lets the full sequence — job selection → CV tailoring → grounding validation → form completion → automatic submission → confirmation → status tracking — run end-to-end and be demonstrated live: *AI finds job → AI decides → AI creates CV → AI fills application → AI submits → application appears in tracker*, entirely within CareerForge's own infrastructure. The mock ATS is treated as a Tier 1 integration in every respect (same Submission Router path, same idempotency and failure-handling behavior, Section 7.2) so the demonstration exercises real code, not a scripted facade.

17. Mobile App (React Native / Expo) Implementation Notes

The mobile app is a genuinely separate client — its own repo/package, its own release cadence through the App Store/Play Store, and its own set of platform concerns that the web app doesn't have. The backend API contract is stabilized before full mobile implementation: mobile development begins once the core API contracts for M1–M4 are stable, so mobile screens are built incrementally against a settled contract rather than duplicating the entire web implementation at the end (Section 19).

> **Scope note:** the mobile application provides the core user experience required for the capstone, prioritizing Profile, Careers, Roadmap, Jobs, CV/Portfolio, application review (UC-8), and application-status tracking (UC-9). Advanced administrative functions (Section 5.10, UC-10 to UC-12) remain web-only — building a full admin experience twice would make the project's scope unrealistic for a capstone timeline without adding proportional value, since the Admin actor is not a mobile-first user.

17.1 Navigation

React Navigation, structured as a bottom-tab navigator for the core student-facing areas (Profile, Careers, Roadmap, Jobs, CV/Portfolio) with a native stack navigator nested inside each tab for drill-down screens (e.g. Jobs list → Job detail → Review/Approve Application → Apply).

17.2 File Upload (CV)

CV upload uses `expo-document-picker` to select a PDF/DOCX from device storage (or `expo-image-picker` if a user photographs a printed CV), then uploads it as `multipart/form-data` to the same `POST /api/v1/cv` endpoint the web app uses — no mobile-specific upload route.

17.3 Push Notifications

Expo Notifications for events worth interrupting the user for: a new job posting matches their target career above a threshold, an application status changes (stretch, Section 5.7), or a roadmap milestone is due. Device push tokens are stored per-user and are opt-in.

17.4 Deep Linking

A universal/app link scheme (`careerforge://job/:id`, resolving to `https://careerforge.app/jobs/:id` on web) so a push notification or a shared portfolio link opens directly to the right screen in the app if installed, or the web page otherwise.

17.5 Offline & Poor-Network Handling

- Already-fetched profile, roadmap, and job-list data is cached client-side (e.g. via a query library with persistence) so the app remains browsable — not blank — with no connection.
- Mutations made offline (e.g. marking a roadmap item complete) are queued and retried when connectivity returns, with a visible "pending sync" state rather than a silent failure.
- AI-backed screens (career match, CV analysis) clearly show a loading/retry state rather than a bare spinner, since these calls are the slowest in the system (Section 6, Performance NFR).

18. Testing, CI/CD & Quality Assurance

18.1 Testing Pyramid

Layer	What's covered	Tooling
Unit tests	Scoring functions (skill-gap %, job match %), roadmap generation logic, CV parsing utilities — pure functions with no I/O.	Vitest / Jest
Integration tests	Each `/api/v1/*` route against a real (test) Supabase project: correct data returned, correct status codes, auth rejected when the token is missing/expired.	Jest + Supertest, or Playwright API testing
RLS policy tests	Automated checks that user A truly cannot read/write user B's rows for every table — run directly against Postgres with two seeded test users.	pgTAP or a scripted SQL test suite
End-to-end (web)	Critical flows: sign up → build profile → get career match → upload CV → generate portfolio.	Playwright
End-to-end (mobile)	The same critical flows on a simulator/emulator.	Detox or Maestro
AI evaluation harness	A fixed set of sample profiles with human-reviewed "expected" career/skill-gap output, re-run whenever the prompt or model changes, scored for schema validity and qualitative relevance — not treated as pass/fail unit tests, but tracked over time.	Custom eval script + versioned fixtures
Anti-fabrication (grounding) tests	A fixed set of profile + job-posting fixtures run through CV/cover-letter generation; every output claim is diffed against the source profile and the test fails on any ungrounded claim. Run on every change to the tailoring prompt (Section 15.5) — this is a hard pass/fail gate, unlike the qualitative AI evaluation harness above.	Custom fact-diff script + versioned fixtures
Submission-tier routing tests	Given a job's platform/ATS source, confirm the Agent selects the correct tier (1/2/3, Section 16.2) and, for Tier 1, exercises the submission call against a mocked ATS API rather than a live one.	Jest + mocked HTTP client
Automatic-application policy tests	Verify the Agent never submits when auto-apply is disabled, when the job is below the user's configured threshold, or when the job matches an exclusion (FR-7.13, FR-7.14).	Jest + fixture profiles/settings
Daily application-limit tests	Verify the Agent stops submitting once `maximum_daily_applications` is reached, and resumes correctly the following period.	Jest
Duplicate-application tests	Verify the same job (or an equivalent posting) cannot be submitted twice unintentionally (FR-7.12).	Jest + seeded duplicate fixtures
Unsupported-platform tests	Verify Tier 2/3 jobs never trigger autonomous submission — only auto-fill followed by a required user confirmation.	Jest + Playwright
Submission-failure tests	Verify that mocked API failures are recorded on `ApplicationAttempt` and do not result in uncontrolled repeated submissions (Section 7.2).	Jest + mocked HTTP failures
Consent tests	Verify that revoking auto-apply immediately prevents further automatic submissions, including for jobs already queued.	Jest
Mock ATS end-to-end test	Verify the complete autonomous workflow — discovery through submission and tracking — against the controlled mock ATS environment (Section 16.2).	Playwright, mock ATS

18.2 CI/CD

GitHub Actions pipeline on every pull request: install → lint/typecheck → unit + integration tests → build. Merges to `main` auto-deploy the web app to a Vercel preview/production environment; Prisma migrations run as an explicit, reviewed pipeline step against the target Supabase project rather than automatically on every deploy. Mobile builds use Expo Application Services (EAS) for beta builds distributed via TestFlight/Play Internal Testing ahead of a full store release.

18.3 Error Monitoring & Observability

Sentry (or equivalent) on both the Next.js backend and the React Native app, capturing unhandled exceptions, failed API calls, and LLM-call failures with enough context (route, user id — never the raw CV content) to debug without exposing personal data in logs.

18.4 Performance Benchmarking

Lightweight load testing (k6 or Artillery) against the standard-CRUD and AI-backed route classes from Section 13.1, run before each major release to confirm the p95 latency targets in Section 6 still hold as the job-postings table grows.

18.5 Security Testing

- RLS policy tests (18.1) run in CI on every schema change — the single highest-value security test given how much this project relies on RLS.
- Dependency vulnerability scanning (e.g. `npm audit` / GitHub Dependabot) on every PR.
- Before public release: a focused manual pass against the OWASP API Security Top 10 (broken object-level authorization, excessive data

exposure, and rate-limiting gaps are the three most relevant to this system's shape).
- Grounding-failure logs (Section 15.5) are reviewed regularly, not just alerted on — a rising failure rate from one job platform is a signal worth investigating even though each individual failure was correctly caught.

18.6 Usability Study

A short moderated usability session with 5–8 target users will be conducted after the major student-facing core modules (M1–M7) are integrated end-to-end. The study will evaluate whether users understand career-match explanations, skill gaps, recommended actions, job-match scores, and the application workflow without additional explanation.

18.7 System Success Metrics

Measurable criteria the capstone can be evaluated against, rather than a subjective "does it feel finished." Grouped into technical metrics (how the system behaves) and product-level KPIs (what a user or evaluator would actually check during a demo).

Technical metrics

- **Career Matching:** the system should produce deterministic and reproducible career-match scores for identical profile and career inputs.
- **Job Matching:** the system should produce consistent compatibility scores for identical user/job inputs.
- **API reliability:** standard CRUD endpoints should meet the defined p95 latency target (Section 6) under the selected test load.
- **AI reliability:** AI outputs must pass the defined schema-validation tests (Section 15.1) before persistence.

Product-level KPIs

- ≥ 90% of generated CV factual claims pass grounding validation on the first attempt (Section 15.5) — a lower rate signals a prompt problem, not just an expected failure rate.
- 100% of automatic submissions respect the user's configured limits and exclusions (FR-7.13, FR-7.14) — this one is a hard 100%, not a target, since it's a safety property (Section 18.7, Grounding row).
- 0 duplicate submissions in controlled testing (FR-7.12).
- ≥ 95% successful API requests under normal test load (Section 18.4 performance benchmark).
- ≥ 80% of usability-test participants (Section 18.6) understand why a job or career was recommended, without additional explanation.
- An application can travel end-to-end — job selection → tailoring → grounding → form completion → Mock ATS submission → tracker — in a single observable run (Section 3.5), demonstrating the full pipeline rather than its parts in isolation.

18.8 Production Verification Checklist

Before Phase 10 (Section 19) is considered complete, the team runs this checklist against the live deployment — it is the deployment-time counterpart to the module-level acceptance criteria in Section 5.11, and what "smoke testing" and "monitoring verification" concretely mean in the roadmap:

1. User registration and authentication.
2. Profile creation and retrieval.
3. Career and skill analysis (deterministic score + LLM explanation, Section 15.0).
4. Job discovery and deterministic matching.
5. CV analysis and portfolio generation.
6. AI-generated CV/cover-letter grounding validation (Section 15.5).
7. Application Agent eligibility and policy checks (Section 7.2).
8. Mock ATS application submission (Section 16.2).
9. Application tracking and status updates (UC-9).
10. Mobile client communication with the production API.
11. Authentication, authorization, and RLS verification (Section 14).
12. Error handling and logging — Sentry/observability confirmed active (Section 18.3).

A deployment is considered successful only when every item above operates correctly against the production environment, not merely against local development.

19. Development Roadmap

Phase	Focus	Deliverables
0	Engineering Foundation	Git repository initialized and connected to GitHub; Next.js + TypeScript web project scaffolded; React Native + Expo mobile project scaffolded; Supabase development project created; Prisma schema initialized; environment-management strategy established (`.env` per environment, no secrets committed); GitHub Actions CI pipeline configured (see Section 18.2).
1	M1 Profile & Onboarding	Supabase Auth wired to `auth.users` , `profiles` /education/experience/project/skill CRUD, RLS policies on all user-owned tables (Section 9, 14.2).
2	M2 Career & Skill Analysis	AI Integration Layer (Section 15), career-match endpoint, deterministic skill-gap scoring with LLM explanation, persisted `CareerMatch` records with prompt/model version.
3	M3 Academic Planner	Course/UserCourse CRUD, degree-progress calculation, career-driven course recommendations.
4	M4 Job Discovery & Matching	Job-postings dataset sourced per Section 16 (admin-curated for the demo), deterministic compatibility scoring engine, market skill-frequency aggregation, stale/expired posting exclusion (FR-4.5). Mobile development begins once M1–M4 API contracts are stable (Section 17).
5	M5 Learning Roadmap & Project Recommendations	Roadmap generator and project-recommendation endpoint.
6	M6 CV Analyzer & Portfolio Builder	Baseline CV upload + parsing + deterministic scoring + LLM feedback, versioned CV storage (FR-6.6), portfolio generator with templates.
7	M7 AI Application Agent	All nine internal components (Section 7.2): eligibility filter, policy engine, CV tailoring with the anti-fabrication grounding check (Section 15.5), cover-letter generation, form auto-fill, Submission Router (Tier 2 assisted-apply plus one controlled Tier 1 integration against a sandbox/mock ATS, Section 16.2), application tracker, and the outcome-analysis feedback loop (FR-7.8).
8	Web/Mobile Integration	React Native screens for the core student-facing modules (Section 17) against the now-stable API; Supabase authentication integrated; native navigation and document/file selection completed; review/approve-application flow (UC-8) on mobile. Admin Dashboard (Section 5.10) remains web-only by design.
9	Testing, Security & AI Evaluation	Unit, integration and E2E tests; anti-fabrication, submission-tier, policy, duplicate, and consent tests (Section 18.1); security tests; AI evaluation; performance testing; mobile compatibility testing (see Section 18).
10	Deployment & Demonstration	**Production deployment:** production Supabase environment prepared, database migrations validated and applied, Next.js web/API deployed to Vercel, React Native production builds generated with EAS. **Smoke testing:** the production verification checklist (Section 18.8) — registration, core workflow, Mock ATS submission, mobile connectivity — executed against the live deployment. **Monitoring verification:** Sentry error capture and the logging/observability setup (Section 18.3) confirmed active in production before sign-off. Mock-ATS demo scenario rehearsed (Section 16.2); v1.0.0 Git release tagged only once all three deliverables pass.
11 **stretch**	Post-v1.0.0 Enhancements	M8 course-failure simulator, M9 conversational AI assistant (Section 4); expanded Tier 1 ATS integrations beyond the first one built in Phase 7; see also Section 21, Future Enhancements.

20. Getting Started — How to Build/Run

1. **Create the Supabase project.** New project at supabase.com → note the project URL, anon key, service role key, and the Postgres connection string.
2. **Scaffold the web app.** `npx create-next-app@latest --typescript`, add Tailwind CSS, install `@supabase/supabase-js` and `@supabase/ssr` for client/server Supabase access, and `prisma` + `@prisma/client`.
3. **Point Prisma at Supabase.** Set `DATABASE_URL` (and a separate `DIRECT_URL` for migrations) to the Supabase Postgres connection string in `.env`, model the schema from Section 9 in `schema.prisma`, then run `npx prisma migrate dev`.
4. **Enable Row Level Security.** For every user-owned table, turn on RLS in the Supabase dashboard and add an owner policy (see the example in Section 8.1) for select/insert/update/delete.
5. **Wire up Auth.** Configure Supabase Auth (email/password + any OAuth providers), and verify the session JWT server-side in a shared Next.js middleware/helper used by every API route.
6. **Add the LLM integration.** Create one server-only module (e.g. `lib/ai/client.ts`) that wraps the LLM API call, so every AI-backed v1 route (career-match explanation, CV feedback, cover-letter/CV tailoring) shares the same prompt-building and error-handling logic. The AI assistant (M9) reuses this module later — it is not part of the v1 build.
7. **Build modules in the Section 19 development roadmap order.** M1 → M2 → M3 → M4 → M5 → M6 → M7 → Web/Mobile Integration → Testing, Security & AI Evaluation → Deployment & Demonstration, each module with its own API routes (Section 13), UI pages, and at least one integration test per endpoint.
8. **Scaffold React Native.** `npx create-expo-app careerforge-mobile --template`, add React Navigation, an HTTP client (e.g. Axios/fetch) pointed at the deployed `/api/v1` base URL (or the Supabase JS SDK for auth/storage convenience), and build the core student-facing screens (Section 17) against the stable API.
9. **Deploy.** Push the Next.js app to Vercel, set the same environment variables (Supabase URL/keys, LLM API key, `DATABASE_URL`) in the Vercel project settings, run the Prisma migration against the production Supabase database, and deploy the Mock ATS alongside it before first release (Section 19, Phase 10).

21. Future Enhancements

Beyond the stretch modules already modeled in Section 4 (M8–M9), a few enhancements are worth planning for even though they sit outside the capstone's v1 scope:

- **Dedicated market-analytics dashboard.** The Admin's "View Platform Analytics" use case (UC-11, Section 10) currently covers basic visibility; a richer version would chart skill-demand trends over time (not just a point-in-time snapshot) and career-match distribution across the user base.
- **Cross-user outcome feedback.** M7's feedback loop (FR-7.8) already adapts each student's own recommendations from their own outcomes; a future version could correlate CV-analysis scores with actual interview rates across the full (anonymized, aggregated) user base to sharpen the "why was my response rate low" reasoning platform-wide.
- **More Tier 1 ATS integrations.** v1 ships with one controlled Tier 1 integration (Phase 7, Section 19); expanding the list of platforms the Agent can submit to directly — versus Tier 2 assisted — is ongoing work, not a one-time milestone.
- **Public job-board API ingestion.** Moving beyond the admin-curated v1 dataset (Section 16.1) once a provider's terms, rate limits, and access requirements have been verified.
- **Expanded AI evaluation.** The evaluation harness in Section 18.1 starts with a small fixed fixture set; growing it with real (anonymized, opt-in) student profiles over time would make prompt regressions much easier to catch — for the anti-fabrication tests (Section 15.5) especially.
- **Continuous usability research.** Section 18.6 proposes one study after core-module integration; running a lightweight version each semester would catch UX drift as modules are added.

None of these block a v1 release — they are listed so the team can make a deliberate choice about what to defer, rather than discovering the gap late.

21.1 System Limitations

> CareerForge does not guarantee employment or universal automatic submission across all job websites. Autonomous submission depends on the availability of a permitted and technically supported API or integration. Unsupported platforms are handled through assisted or manual workflows (Section 16.2). The system is designed to automate the career and application process as far as the available technical and platform constraints allow — not further.

22. Risks & Mitigation

Most of these mitigations are mechanisms this report already specifies elsewhere; collecting them in one table makes the engineering analysis explicit rather than implicit.

Risk	Mitigation
LLM hallucination / fabricated applicant facts	Grounding validation gate — every CV/cover-letter claim fact-diffed against the verified profile before persistence (Section 15.5).
Job-board API unavailable or ToS-restricted	Admin-curated dataset and Mock ATS as the v1 source of truth (Section 3.4–3.5, 16.1); external APIs are an opt-in future enhancement, not a dependency.
Duplicate applications	Duplicate-application detection in the eligibility filter (FR-7.12) plus a database-level UNIQUE constraint as a backstop (Section 9.4).
Submission API failure (Tier 1)	Failure handling records the reason on `ApplicationAttempt`, avoids uncontrolled retries, and routes to a review queue (Section 7.2).
Duplicate submission from network retry	Idempotency keys on every submission request (Section 7.2).
LLM API outage or latency spike	Cached last-known results served where available (Section 15.4); AI-backed routes show a clear retry state rather than blocking the page (Section 6, Reliability NFR).
Excessive/uncontrolled automatic applications	User-configurable daily limits and minimum match-score threshold, enforced in code and at the database layer (FR-7.13, Section 9.4).
User revokes auto-apply mid-flight	Consent is checked immediately before submission, not only when the setting was last saved (Section 6, Consent management).
Platform blocks or prohibits automation	Submission Router defaults to Tier 2 (assisted) or Tier 3 (manual bundle) for any platform without a supported API — never attempts to force Tier 1 behavior (Section 16.2).
Stale/expired job postings acted on	Freshness check (`is_active` , `expires_at`) re-verified by the eligibility filter immediately before acting, not only at discovery time (FR-4.5, Section 7.2).
Sensitive application questions answered incorrectly	Uncertain/sensitive fields are never inferred — the Agent leaves them for explicit user input rather than guessing (Section 15.5).
Scope creep beyond a capstone timeline	Explicit MVP scope boundary (Section 3.4): one Tier 1 integration, core modules only, stretch modules deferred (Section 21).
Credential/secret leakage	Server-only secrets, RLS as a second access-control layer, encrypted third-party OAuth tokens (Section 14).

23. Conclusion

CareerForge is an AI-powered career and application automation platform that connects a student's verified profile, career analysis, skill-gap assessment, academic planning, job matching, learning roadmap, CV/portfolio generation, and application management into one closed-loop system. Its primary differentiator is the **AI Application Agent (M7)**, which can autonomously identify suitable jobs, tailor application materials, complete application forms, and submit applications through supported integrations when authorized by the user. Where automatic submission is not technically or contractually available, the system switches to an assisted workflow requiring explicit user confirmation (Section 16.2).

The system therefore goes beyond recommending what a student should do: it converts recommendations into controlled actions while maintaining user consent, factual grounding, application limits, auditability, and platform compliance (Sections 6, 15.5, 21.1). Every score a student sees is computed deterministically and explained by the LLM, not invented by it; every generated document is grounded in verified data or the job posting itself; and every automatic action stays inside limits the student configured and can revoke at any time.

The seven core modules (Profile & Onboarding, Career & Skill Analysis, Academic Planner, Job Discovery & Matching, Learning Roadmap, CV Analyzer + Portfolio Builder, and the AI Application Agent), together with the Admin Dashboard (Section 5.10) that keeps the matching engine fed, are enough on their own to demonstrate full-stack development, relational data modeling, authentication and authorization, third-party AI integration, and a working mobile client — while the two remaining stretch modules (Course-Failure Simulator, AI Assistant) give clear, already-modeled room to extend the project further if time allows.

This report is a starting design document, not a frozen specification — expect the schema and API surface to evolve slightly once implementation begins, particularly around exactly which fields the LLM prompts need, how job-posting data is sourced, and which Tier 1 ATS (or mock ATS, Section 16.2) integration is built first.